



# AWS Certified DevOps Engineer - Professional (DOP-C02): SDLC Automation

## CodeCommit

## CI/CD Concepts

Hey, Cloud Gurus. Welcome to the first lesson in this course and the first lesson in domain one, SDLC Automation, and that is CI/CD Concepts. So what are we going to do in this lesson? Well, first of all, we're going to define the problems that we encounter. And ultimately, we want to have automation. So we're going to have to overcome some hurdles to get that automation. We'll talk about the problems presented to us and how we solve those problems. Then we're going to talk about dev and operations and their goals. But our goal is to merge these goals into one, what we obviously call DevOps. Then we'll talk about solving some of the problems that we've defined already in the lesson. We'll talk about DevOps in action and then a very important concept, CI/CD. We'll define CI/CD. But we have two terms, continuous delivery and also continuous deployment. What are the differences between the two? Then we'll talk about developer tools. So this gets AWS-specific. Are tools such as CodeCommit, CodePipeline, CodeDeploy AWS-specific developer tools to help us create CI/CD pipelines? And then we'll go over some exam tips. So this is a big lesson, very important lesson to set the foundation for this domain. So let's go ahead and talk about some of the problems that we encounter. Now before we get into CI/CD, let's just talk about general software development and some of the problems we're trying to solve. And I'll focus on the old-school waterfall software development projects. These types of projects are typically slow, not iterative, and have long release cycles. And we basically want the opposite. We want fast, iterative, short release cycles. So that's the problem that we want to solve. How do we do that? And let's just emphasize this point. And this is from the field, actually an experience of mine. There was an educational support company who needed to redo their application. The application was an absence management system for school systems, a very large application. They needed to redo it. And most importantly, they needed to beat their chief competitor to market. So they hired a company, and they gave the company a bunch of requirements and some cash. Now that draws a picture of bags of cash, but maybe a slight embellishment, but giving them a lot of money. Software development team goes away for six months and develops a first cut of this application. Well, it's all there. They come back, and they demo it to their client. This thing is woefully slow and has many, many bugs in it. And that first six months really put the whole project behind the eight



ball, and it was time and money, everything was slipping, and ultimately that project failed. What could have possibly gone wrong? Well again, they started out with a bunch of requirements, and notice the cash and requirements, I have a big arrow. And other great things, testing, adaptability, accountability, sort of a small arrow. What I'm getting at, it wasn't an iterative process. There weren't short release cycles, frequent feedback, none of that occurred, and that's why it failed. Now one thing that could've helped was an agile software development process. DevOps, in fact, shares many characteristics to agile software development. And of course, there was a development team and an operations team in this whole picture. Well, guess what? This might be captain obvious, but dev and ops don't share the same goals. The focus of the development team is, hey, how many changes, features, and defects can we push into production and how quickly can we do it? Operations, well we want stable and available systems, and those goals don't align too greatly. So how do we sort this out? Well, who's driving all of this, is it dev or ops? Ah, neither really. It's the business. Does the business want rapid releases? They sure do. Or do they want a stable system? Well, they want that too. How about a little bit of both? So there's our problem. We want rapid releases and we want to do it in a stable manner. This is dev, this is ops. Ultimately, we need to merge these goals, dev and ops. And guess what? Hey, why don't we test this out as well? Let's bring QA into this picture. And this is how dev and ops came about, merging the goal of dev and ops, adding in some testing. Well, they have their goals too. Let's merge those goals in as well. In the middle here, we have DevOps. Ultimately, we want to shorten our dev lifecycle and deliver our features, fixes, and updates frequently. And that's going to be a big focus of this section. And a key part of all of this is automation. Rapid releases, we don't want to be doing this manually all the time. We want to automate all of this. All right, so this sounds great. But how does this look in action? All right, we have our cool figure eight here, and it's also the symbol for infinity and continuity. Continuous, well, continuous integration, continuous delivery, which we'll talk about here briefly. So where do we start? Well, we plan, we code, we build, we test, we release, deploy, operate, and then we monitor, and we keep doing this. The continuous process is driven by the business. We're getting maybe some bugs from our clients. We're getting some feature requests from our requirements. We want to do deployments rapidly, but stably in a continuous process. So what is CI/CD? Well, the CI is continuous integration. As we can see, our system is feeding back into itself. What is CD? Continuous delivery. You're probably thinking, okay, I've heard of that, but I've also heard of continuous deployment. Is there a difference? Are they the same? The answer to that is no, they are not the same. So we have our source code, and we test and build our code. QA gets into the picture. We do some more testing, and then we release. So we have continuous integration and continuous delivery. Now watch what happens here. Ah, we have a stop sign for continuous delivery. Well, that's not very continuous, right? Well, we're automating all the way out to the point before releasing to prod. We don't want to do that just yet. We want to perform some checks before we go out to prod. Then we deploy to prod. Now watch this.



Continuous deployment is a full automation from end to end. We are deploying the whole way out to prod. We must have great confidence in our process. No fear of our deployment crashing. No fear of bugs. We have this thing down pat. That's the key difference right there. Continuous delivery has that last sort of failsafe check before we deploy. Continuous deployment goes straight out. One way I remember that. When I think of delivery, I think of a mailman delivering mail to our box. We have to run out and get it. So it doesn't come straight to us. Whereas continuous deployment, maybe think of that as Gmail, it just comes right to us. All right, this all sounds great. Now what about the tools to perform our automations? Well, this is an AWS course. So let's focus a bit on developer tools on AWS. AWS has the Developer Tools suite, and these aren't the only tools. But CodeCommit for a repository, think about GitHub, and we'll talk about the differences in an upcoming lesson. CodeBuild, we need to build our code. When I think of building code, I think of Java, and the end result of a build is a JAR file, and that's what CodeBuild is going to produce for us. So we have our build, we need to deploy it. Well, AWS CodeDeploy can handle that for us. How do we manage and automate all of this? We can do it with CodePipeline. And that's a brief overview of these services. We'll certainly have in-depth lessons on each of these. All right, let's review a few elements from this lesson and offer some exam tips. So we covered CI/CD and the differences between continuous delivery and continuous deployment, dev and ops and their different goals. But we can get these working together and form DevOps. We'll get some help from QA as well. And how does DevOps look in action? Just remember our figure eight symbol for infinity. So it's a continuous integration and a continuous delivery and/or deployment. And of course, we talked about developer tools. Now, just to emphasize the point, what is continuous deployment? Remember, this is going the whole way out to production without the safeguards of continuous delivery. Continuous delivery has that symbolic stop sign. So we want to automate as much as we can, but we're going to have that last safety check before throwing our code over the wall and deploying it on prod. So that's a look at some CI/CD concepts. It's a look at automation and dev and ops. And we'll certainly be building on this as we go throughout the course. So thank you for joining me in this lesson, Cloud Gurus. I will see you in the next lesson.

## CodeCommit Overview

Hello, Cloud Gurus. In this lesson, we're going to take a look at CodeCommit. And remember, this is going to be our repository where we store our code. We're going to take a high-level look at CodeCommit. But don't worry, we'll certainly go deeper in upcoming lessons. So what is CodeCommit? Well, CodeCommit is a managed service that hosts private Git repositories. So we really have two key terms here, managed service and Git repositories. All right, so, when we use CodeCommit, underneath CodeCommit is Git, and we can use Git commands to manage our repository. Now speaking of managed, it is a managed service. And



when you think of managed services, you usually think of RDS where we set up our database, but we have no concerns about what's going on behind the scenes. This is a similar scenario. With CodeCommit, we don't worry about the servers. AWS manages the servers for us, and it manages the capacity of those servers, and our repositories can grow without bounds. So that raises the question, why not just use Git? Well, you can, and you can use several other types of repositories besides CodeCommit. The advantage to CodeCommit is that it is a managed service. So if you're using something else, if you're using GitHub, you have to worry about the servers and your capacity.

With CodeCommit, you don't have to worry about that at all. AWS manages that for us. So let's review some benefits of CodeCommit. We already talked about managed service. It is highly available, scalable, and fault tolerant. Well, that comes into the managed service again. AWS is handling this for us. We don't have to scale, we don't have to make it highly available or fault tolerant, no size limit. So this thing will scale automatically for us without us doing anything. And it integrates with other services. And the key services we are interested in are CodeBuild, CodeDeploy, and CodePipeline, but it also integrates with Lambda and SNS. And those services are always key tools in setting up automations. And that is a big focus of this section of the course and really the entire course. And again, it works with existing Git-based tools. If you have experience with Git, well no problem. This is going to be very familiar to you. And you'll see in an upcoming lesson, the commands are going to be very familiar as well. Now in this section of the course, we're going to create a CodeCommit repository and then perform actions on it. So, we'll go out to the Management Console and create a CodeCommit repository. And then in upcoming lessons, we will work against that repository using CodeCommit commands. But before we do that, let's do a little interactive demo right here. And this will give us a very good understanding of what CodeCommit is doing for us. So let's say that we have our CodeCommit repository, and we have a version 1 of our repository of our code. So we bring a new developer on board, and our new developer wants version 1. And what they would do to get version 1 is use the clone command. That would pull the version 1 down to their local machine. So this developer has a task to do. Maybe let's call it a bug fix. They make the changes to the code. And the next step they need to do is a commit. They need to commit this fix, and that creates a version 2. But notice, the location of that version 2 is still on the developer's local machine. What they need to do next is a push. They are pushing version 2 to the central repository. So it's not going immediately to the central repository. Developer commits it locally, probably does their own testing, could have somebody else test, push it to the central repo. The central repo has a version 2. Now we have another developer, and let's say developer 1 Slacks developer 2, hey, I pushed a new version. You might want to grab that. It's stable. How does developer 2 retrieve that new version? They do a pull from the central repository, and that is going to give them version 2. So that's really an overview of how we're going to use CodeCommit, how we're going to interact with CodeCommit. And again, if you have any experience with GitHub, this is going to look very familiar to you. So now



let's bounce out to the Management Console and set up our CodeCommit repository. All right folks, here we are in the AWS Management Console, and we are in CodeCommit. Now if you go in CodeCommit and you see an introductory screen, well you probably haven't created a repository before. But the key point is that you will definitely see a Create repository button, and that's what we're going to do. So let's go ahead and click Create repository, and we'll give our repository a name. You can call it anything. If you want to follow along exactly, I'm going to name it DevOpsPro. And we can give it a description. We can add some tags and enable code review for Java and Python. And let's go ahead and click Create, and there we go. We have an empty repository. Now we can create files directly in here, and that's probably not what you're going to be doing most of the time. You can go ahead and add files, generally an upload or more probably using the command-line interface to upload files. And we'll work with the command-line interface in the next lesson. All right, so we've talked about in a previous lesson how to set up HTTPS for our Git credentials and our communication with our repository. There's the SSH option, HTTPS(GRC). We're just going to use HTTPS. It's probably the simplest way to do this, but let's go ahead and upload a file to our repository. All right, so I'm just going to upload a script just for demonstration purposes. In an upcoming lesson, we'll actually upload some real code. (Working) You have to enter an author name, email address, and optional message. And note here, nothing happens until we commit changes. And the commit command is going to be a key command when we're working with CodeCommit. And there we go. That gives an idea of how we can use CodeCommit from the Management Console. And again, we'll start working with this repository from the command-line interface. Just to review, let's take a look at some exam tips. So CodeCommit has many benefits. And the big one is that it is a managed service. We don't have to provision servers. We don't have to scale servers, make them fault tolerant. AWS is doing all of that for us, but it is not the only repository you can use in an AWS deployment pipeline. This is an exam tip that has helped me many times. So when in CodeCommit or CodeBuild, CodeDeploy, and especially tying it all together, CodePipeline, when you're setting up a repo or you're setting up a build or deploy or pipeline, as you go through setting it up, you're going to have drop-downs. And those drop-downs can be great study tools. So, for example, we will see this in upcoming lessons. But when we set up our pipeline, we're going to have those drop-downs, and it's going to tell us what is available in the build section. It could be CodeBuild. We'll find that it can be Jenkins. Likewise, with CodeDeploy, those drop-downs are going to tell us the services that can work with CodePipeline. So it's a pretty tricky study tool that's very helpful. Now we mentioned that we can use other repositories in our AWS pipelines, and these include S3, GitHub, Bitbucket, and GitHub Enterprise. Now think about S3, and this is a professional level course, and you probably recall this from maybe previous learning. What do we need to do to make S3 a repository? And the answer is we need to turn on versioning. And that makes perfect sense when you think about how a repository operates. So just remember that key point about S3. Turn on versioning, it will become a



repository. And in fact, if you try to use S3 as a repository and you don't have versioning on, you will be told in the console to turn on versioning. So that is an overview look at CodeCommit. Key points to remember, this is a managed service. And also, other repositories can be used in our deployment pipelines. And with that, Cloud Gurus, I'm going to end this lesson. Thank you for joining me. I will see you in the next lesson.

## CodeCommit Repository Actions - Demo

Hey, Cloud Gurus. Welcome this lesson, CodeCommit Repository Actions. Now the AWS Management Console is a really handy tool. But as a DevOps engineer, you'll be working often from the command-line interface. And in this lesson, we'll take a look at some of the really key commands that we can perform with CodeCommit from the command-line interface. And those commands are Create. So we'll create a CodeCommit repository from the CLI. We can view a repository from the CLI. We can list all of our repositories from the CLI, and we can delete from the CLI. Now we'll also bring up the Management Console to verify some of our work. But these are really key commands. And remember, anything that you can execute from the command line, you can script as well. So we always think about automation, and this is a stepping stone toward automation. We'll finish up the lesson with some exam tips. So let's go ahead and take a look at working with CodeCommit and doing repository actions from the command-line interface. So looking at the diagram, we're going to be operating as a developer, issuing commands to CodeCommit. Now I've already created an EC2 instance, and we're going to use that instance and issue commands to CodeCommit. And this really simulates what the developer would be doing in a normal dev environment. Now before we jump out to the demo, let's just take a look at some of the commands. We're going to create a repository. We'll actually create a couple repositories. We can view a repository, list our repositories. And at the end, we're going to clean up after ourselves. We will delete a couple repositories. So let's just jump out to the demo and see what we can do with these repository commands. All right, let's dive right in and start working with these repository commands. So I have three tabs open, and we want to be in CodeCommit in the Management Console. And we see the repo we created in the last lesson. I also have another repo that I work with. Now before we do anything with CLI commands, remember that we have to be authenticated. So let's go over to IAM, and I'm in Users. And I actually selected one of the users. Go to the Security credentials tab, and we need some access keys. So just click Create access key, and then you can download the access keys. Now remember, best practice, when you're not using access keys, don't leave them hanging around. So when you're done with them, go ahead and delete those. So once we have access keys, we want to work from the command line, and I'm going to use CloudShell. So you can type that in your search bar and get to CloudShell. Just increase the font size a bit. So, we have our access keys. We've downloaded



those, and then we would run `aws configure`. Now notice, I've already put in my access keys, so I'm not going to do that again. I can just click Enter, but here's where you would plug in your access key and your secret access key. Make sure the region name is `us-east-1`, and it defaults to an output format of JSON. Just going to leave that, and we're ready to go. So now we can start issuing commands using the CLI to our account and specifically CodeCommit. We set our context `aws codecommit`, and let's list our repositories. Now we just saw that I have two. So, there we go. That lines up. So we're interacting with CodeCommit from the command line. Now let's go ahead and create a repository, and you'll find that most of these commands are very literal. Want to create a repository? Well, the command is `create-repository`. We need to give our repository a name, and I'll just call it `DummyRepo`. I'm just going to delete this at the end of the lesson. There's our command. All right, we got back some JSON, don't see an error message. Let's trust, but verify. There's our new `DummyRepo`. I'll just clear the screen and give us a little bit more room. All right, let's say we want to check out a particular repository. We can do `get-repository`. And of course, we have to provide a name, and we're interested in the repo that we're going to use throughout the course, so let's see how this works. There we go, `accountId`, `repositoryId`, `repositoryName`, there's a description, `defaultBranch`. We'll talk about branching in upcoming lessons. Notice, `cloneUrl`. I'm going to copy that to the clipboard. What if we want to clone this repo? We would need this URL. Now I'm going to hold off on that. We'll do that in the next lesson, but it's good to point that out. And notice, there's an HTTP `cloneUrl` and an SSH. We're interested in the HTTP. We'll do that in the next lesson. All right, and just to remind us where we're at, let's list our repositories again. So we have three. Let's delete that `DummyRepo`. The command goes a little something like this. As you might guess, if you want to delete a repository, that command is `delete-repository`. And of course, we need to provide the name, `DummyRepo1`. Make sure you get the capitalization right, and let's run that. Looks like it ran okay. Let's verify. Voila, no more `DummyRepo`. One last check. Let's refresh, no more `DummyRepo`. So those are some of the key commands that we can run on our repos from the command-line interface. And remember, anything you can run from the command-line interface, you can script and have automation. All right, exam tips. And actually, we sort of ended with an exam tip. Anything we can run from the command line, we can script, and scripting is very important for automation. Now we talked about these commands being very literal. So, if asked to select the proper command, note that the majority of these commands are very literal, `delete repository`, `delete-repository`. Now that's not 100% true, but I would say when in doubt, figure it out like that. Hey, I need to delete a repository. Well, let's try, and most likely, it is `delete-repository`. And of course, in this case, that's what it is. Quick command review. There we go. Very literal. This one, of course, is not, and just note the distinction. It is actually `get`, not `git`, `g`, `i`, `t`. So just remember that. `List-repository`, again, very literal. And finally, `delete-repository`. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.



# CodeCommit Cloning, Commit, Pushes, and Pulls - Demo

Hello, Cloud Gurus. In this lesson, we're going to begin working with some of the main Git commands we've been talking about. We're going to do some cloning, some commits, pushes, and pulls. And we will be working with two new users named Cael and Eva. They are software developers, and they need to get their work done using CodeCommit. So we'll simulate Cael doing some work, and he will then do a commit and a push. And Eva is going to want this update, so she will do a pull. Before we begin, let's take a look at what we're doing in our diagram. And this is a refresher. We've seen this diagram before. First of all, our developers are going to want to clone the repository. Now in this particular case, Cael clones a repository, does some work, and then he does a commit. And finally, when all is well, he'll do a push back to the central repository, then we'll have version 2. So we have the clone command. We're going to execute that, a commit, and a push. Now Eva comes into work the next morning. What does she want to do? She wants that latest, so she's going to do a pull. Her machine had version 1. It will then have version 2, and it's really that simple. And of course, this can work in the opposite direction as well. Eva could be doing the commit. Cael wants the latest update. He'll do a pull after Eva does a push. All right, so with all that said, let's work with our command-line interface and do some common Git commands. And of course, we'll go out to the Management Console as well and verify everything. All right, so I'm logged in as an administrator. First thing I want to do is go to Users. And for the user Cael, Cael is going to be issuing Git commands. So we go into Security credentials and just scroll down a bit. This looks familiar, SSH, keys for CodeCommit. We're using HTTPS, and we can generate these credentials. I'm not going to do that. I've already done that. But basically, it's just like access keys. You generate the credentials, and then you can download them. We'll need those credentials when we are working from the command line as Cael. All right, so I'm logged in as Cael. And I want to go to CloudShell, and you can search for that if you're not seeing it on the front panel. And I just want to increase the font size. I'll do it. Now Cael wants to start working with our DevOps Pro repository. So what does he need to do? He needs to clone that repository. Now I've opened another tab, and I've gone to CodeCommit. Cael wants to clone that. Well, we need this Clone URL. So click to copy that. So our command is `git clone`, and then, of course, our HTTPS URL. And let's call our directory `local`. That's where we're cloning to. Now we need to enter that username. All right, looks promising. Okay, so that finished. We do an `ls`. There's our local folder. Let's `cd` into `local`, and `ls` again. There's that `build.gradle` file, which we uploaded when we initially created that repo. So we have successfully cloned that repository to our local machine. All right, I'm just going to clear the screen, and let's use `vim` to edit that file. So I've just added a comment. But hey, guess what? That file is different. So just for demonstration purposes, we'll just use the comment. We changed our file, and let's use our imagination. Maybe we did some major changes. We need to push this file. So I'm going to save this



file. That sequence was escape wq!, saved it with a write. Now, before we do a commit, let's check the status of our repository, git status, and notice in red, modified our build.gradle file. So it's saying your branch is up to date with origin/main. In other words, the main branch has not been changed, and, of course, we have a change. So the first thing we want to do is a git add. So we're going to do a commit, and we're adding this file that we changed to the commit. No news is good news. Now we can run the commit. All right, so git commit -m for message. And basically, we're just testing this out. All right, so we got an error message. And it's kind of great though. It gave us real detailed instructions. It wants an email and a user. All right, let's try that again. Git commit, -m for message, and our message. One file changed, one insertion. I'll just clear the screen. Let's do a git status again. All right, so everything is up to date. Nothing to commit. And of course, we need nothing from the main repository. All right, let's try some more commands relevant to what we're doing. Git remote. Okay, this is where we're at. This is where we're working. We're working from our local, that's our origin. Now let's run git branch. All right, main. So, we're working locally, and the branch that we're pulling from, we did a clone from main. Now think about this. We've done a commit. Are we done? Well, if you read the fine print, notice "Your branch is ahead of origin/main by 1 commit." So we committed, but realize we committed locally. To get it to main, we have to do that push, but not just yet. Let's do this, diff origin/main. So that's pretty descriptive. It's going to give us the difference between our local repository and main. And of course, we know what that is. We change that build.gradle file. Just added a comment. All right, let's do our push, git push. What are we pushing? The origin. Where are we pushing to? We're pushing to main. Let's run that. All right, it wants those credentials again. So I enter that username and password again and did the push. We didn't get an error message, so that's very promising. All right, let's wrap up by going to the Management Console. I'm just verifying this. So we go to CodeCommit, and I'm in Repositories. Well, guess what? Right there is our proof, Last modified 2 minutes ago. There's our build.gradle file. It should have a comment in it, right? There we go. So we have successfully cloned this repository, and we made some changes. Did a commit. Well, that commit is just local. Remember that we still have to do a push. So, with that said, let's take a look at some exam tips. And again, understand that the commit is committing changes to your local repository. To get these changes to the main repository, we need to do a push. It's key to understand that. So our order of operations, we make changes to local files, commit changes to our local repository, and then we push those changes to a remote repository. Now, additional step for somebody else. They already have the repository, they would do a pull from the main repository to get those updates. So with that said, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeCommit Merging and Branching - Demo



Hey, Cloud Gurus. Welcome to this lesson, CodeCommit Merging and Branching. So why do we need to merge and branch our code? Well, we can have conflicts in our code, and we'll talk about that first. And in a nutshell, it's when we have multiple developers working on a file at the same time. We could have some collisions here, we could have some conflict. So we're going to talk about that and conflict resolution. It's always a good life skill to have. And then we'll talk about branching, and this gives us the ability to sort of avoid that conflict. We can create a branch and work on our code fixes in isolation from other code. So merging and branching goes hand in hand, and we'll see how that works out for us. And finally, we'll wrap up with some exam tips. So with all that said, let's go ahead and talk about CodeCommit Merging and Branching. So let's see what that looks like. So our two developers have the latest version of our repository, version 2. And again, they make changes to the same file. Cael pushes first, and that is version 3. Kara then does a push and is notified, hey, Kara, hold on a second. We have a conflict. And furthermore, there is content that you don't have locally, that content being Cael's push. So how do we resolve this? And it's called merging it. There's not actually a merge command, but when Kara goes into the file, she will see both updates. She will see Cael's update and her update. Two developers need to interact and decide which update to keep, and then they delete the update that they don't want to keep. So let's say that they decide to keep Kara's version. Kara would see both of those updates. She would also see Cael's. She would delete Cael's. So again, we see the problem. There's a conflict. The merge process, notice it's not a command, it's just a process will help to resolve these conflicts and keep our team on the same page and also our code. So, again, the team decides to keep Kara's change. We delete Cael's change from the file. Kara just has to do a commit again and then a push back to the master. And you might be thinking, well, what about Cael? Does Cael have that new version, Kara's version? The answer is no. How does he get it? Well, let's think about this. We know how this is done. We've done this already. Cael just needs to do a pull from the master, and he will get that latest version. All right, so that's sort of a reactive way that we can resolve conflicts in our files, and there's the merge process. How can we be proactive? Well, the proactive way we can do this is to create a branch. So let's say that Kara wants to work on updates. She would create her branch. Cael wants to work on updates. He would create his own branch. So in essence, branches are an independent line of development, and they allow us to make changes to our code without affecting the main branch, the main repository. All right, so we know what a branch is. Why would we use a branch? To build a new feature or to fix bugs, maybe both, without disrupting the master. And also, it's another layer between our developers. They could still run into conflicts if you think about it. If they're working on this, again, the same file, the same version of that file, they could still have conflicts. So that merge process is still in play. But this is a little bit cleaner approach with each developer having their own version and taking more of a big-picture approach. Now let's see this a little bit in our diagram. All right, so there's the central repo, and let's say this is Kara's local repository. She already has the



latest version of the code. But what you can do is take this latest version, create a branch, and make changes to that branch. It could be one change, it could be dozens of changes, but when she's done, commit those changes and then push back to the master. And again, the key point to remember about this is, hey, maybe Kara was assigned a new feature to work on. So she creates her own branch, doesn't disrupt the master. She builds the new feature. Here's a key point, that new branch can be tested. Again, we don't want to disrupt the master. We're testing this new branch. Everything checks out. We push back to the central repo, and the central repo would then have version 3. Same situation with bug fixes. All right, so demo time. Let's see how this looks in action. All right, so I'm SSHed in as Cael. Now the scenario is that Cael wants to do some work, and he's going to create a branch in which to do that work. And we're going to use some command-line commands. These aren't crucial for the exam, but I will include them in the lesson materials. It's most important just to understand the concepts of merging and branching, and these demos help ingrain those concepts. So let's first get our bearings. That's where we're located. What do we have in this directory? There's our master. Let's cd into master. We want to branch off of the master. Now, to perform a branch, the command is git branch, but there's another command. It is git checkout, and the git checkout command not only creates the new branch, but it switches into that new repository. So let's do that. All right, so the -b switch specifies we're creating a branch, and then cloudformation is the name of that branch. Cael's just going to work with some CloudFormation templates. So, we created the branch and switched to that new branch. So Cael is going to create a JSON file and then just go ahead and edit it directly. So it's just the JSON file, it could be any kind of file. This is for canary testing, and we can just exit and save. All right, so we have two branches. We have main, and we have cloudformation. Now we can switch between these two, and let's just switch back to cloudformation. Let's just make a simple change to this file. I just changed the name of the file to a second ZIP file. All right, so Cael has edited the canary.json file, and he wants to add that back to the main. It's basically going to be a merge. So we've seen this command before. We are adding a file to our commit, and we can add more than one file certainly. But we're only going to do one file. So we're ready to commit. (Working) So our command is git commit -m, the m is for message and just a short message. All right, one file changed. And that's what we expect. Now this is different. This is where what we expect is not what we're going to do. We would expect to do a push. But for branches, what we need to do is switch back to the main repository. Then from the main branch, we perform a merge. There we go, one file changed. Do an ls. There's our canary. Just to double-check, there's our update, dataaoder2, data loader is misspelled, but dataaoder2. All right, so to recap, Cael had some work to do, did not want to disrupt the master. Cael created a branch, did his work, added the file to his commit, performed a commit. Instead of doing a push with branches, we switch back to our main repository and performed the merge. So Cael is done. Now, what Cael can do, he has no use for the branch anymore. He can go ahead and delete the branch. There's our command, git branch -d cloudformation. Branch is



deleted. All right, so to recap, we talked about merging and resolving conflicts from the CLI, and that's a case where our developers are working on the same file at the same time. So certainly need to learn how to do that. Now, we advanced a step, and we started talking about branching. So this gives the developers sort of their own workspace to develop in. And then when they're ready, they can commit their code, test it, and have it merged back into the main repository. Now it doesn't completely eliminate the concept of having those collisions between two developers. But this is a more formal process, and certainly a development team would have a process when committing branches and merging back to the main repository. And just some communication can really go a long way in avoiding any kind of conflicts that we saw. So to wrap up, three things to remember when it comes to branches. All right, first of all, it just separates the work. So we have our own workspace. Think of the main repository as prod and our branches as development environments. When would we do branching? Well, we have some new features we want to integrate into our prod. Well, we can create a branch, work on and develop those new features when we're ready, perform our commit, test them thoroughly, and then merge back into our main repository. And there we go. We may have developers devoted to bug fixes, the same scenario though. They would just create a branch, work on bug fixes, commit the file or files, and then merge those files back into the main repository. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeCommit Data Security

Hey, Cloud Gurus. Welcome to this lesson, CodeCommit Data Security. Now when we talk about security for CodeCommit in general, if we set up our logins and our policies, that's half the battle. This is the other half. Let's talk about data security in transit and at rest. So, in the lesson, we'll talk about encryption and what we have to do for CodeCommit. And then we'll look at some use cases and how we deal with securing our data for specific use cases. We'll also look at some scenarios, and we will wrap up with some exam tips. So let's talk about CodeCommit Data Security. So looking at our diagram again, the encryption is handled for us, and it uses the key management service. So, in our data, let's use a push as an example. CodeCommit encrypts that stored data as it's stored in the repository. Now the opposite direction, when that data is pulled, CodeCommit decrypts that data. It is pulled from the repository. And think about how we communicate with CodeCommit. Our options are SSH or HTTPS. Well, we have our security there. That data is encrypted in transit. So we have at rest encryption covered, and we have in-transit encryption covered. Now there are some behind the scenes things that we're not really aware of. When we create a CodeCommit repository, AWS is creating an encrypted key. Now I'm not going to do it in this lesson, but we've created a CodeCommit repository. And in the Management Console, if we went to key management service and went into AWS managed



keys, you would see a key for CodeCommit. And certainly take that as an exercise if you'd like to verify that. And remember, we didn't have to do a thing. AWS creates that key for us. It does a lot of work for us. It encrypts and decrypts our data. And just a key point, these KMS keys are by region. So we create a repository in us-east-1. We'll have a key there. If we go to another region, unless we create a repository in that region, we won't have a key there. One other key point. For KMS for CodeCommit, we're dealing with an AWS-managed key. Now you've surely heard of customer-managed keys. We don't use those with CodeCommit. Remember, it's a managed service. It's an AWS-managed key. So that's how we protect our data in transit and at rest. And certainly, AWS plays a big part in this with CodeCommit. And that's the advantage of CodeCommit or one of the advantages. The managed service, much of the security is handled by AWS. But there are some other things we can do. And one of those things is, hey, we need to restrict what can be done to our repositories. How do we do that? Well, I'm showing a deny policy. It certainly doesn't have to be the only option. We could limit some things in our policies and certainly deny certain things that our users can do. And we'll talk a bit more about that momentarily. So, when we think about security and the choice of CodeCommit over other options such as GitHub, well, it's a managed service. And AWS is going to handle encryption in transit and at rest for us. That's a big advantage. There we go. AWS guarantees that for us. And again, it's highly available and scalable. Now, the point of that is we can be sure no matter how large this thing scales for us, that data is going to be encrypted in transit and in rest. So let's take a look at a scenario. So we want our development team to have full access to CodeCommit, but they should not be able to create or delete repositories. We have administrators who do the creation and deletion of repositories. Our developers just need to work with those repositories. So what steps can we take? And that last icon that popped in on our first diagram is a hint. We can add a policy, `AWSCodeCommitPowerUser`. Let's take that a step further. So, we have a dev team. We add this Power User to each of our developers' accounts. Great, right? Well, that would certainly work. What's a better way we can do that? We can create a group for our dev team, probably already have one if we're doing IAM, right? We create a group, we add that policy to our group and port our developers in that group. They then have CodeCommit Power User. And as you might expect, that Power User gives them full privileges in CodeCommit with the exception of creating or deleting repositories. So if Kara from our dev team wants to delete a repository, no sale. She cannot do that, but she can certainly work with it and perform all of the operations. And to summarize, there's our CodeCommit Power User. We attach that policy to each of our developers. Better yet, we have a group for our development team, and we attach that policy to the group. That's the more efficient way to do it. It's less work. What if we have developers leaving, new developers coming on? Well, we just put those developers into our group, and all those permissions are assigned within that group. And when you think about security, that's more fail-safe too rather than working with individual accounts. The more we do this, the more opportunity



there is for error. And just taking it one more step further, as far as security goes, well, we do have those admins and they need full privileges. We could have another group for admins, and those admins, we can attach a policy, `AWSCodeCommitFullAccess`, to that group. And there we have it. We have that separation, and we're securing our CodeCommit repositories properly. And maybe one more thing we can do to sort of granularize our duties here and make this more secure, think about, remember our repositories. The main repository, we think of that as our prod environment, and then we think of our branches as dev. Do we want everybody pushing to our prod environment or doing merges to our prod environment? Probably not. So another way we can restrict things is by adding an appropriate policy, allowing or denying that push capability. All right, so again, I just want to do a quick lesson on securing our data and securing our repositories. And key points, CodeCommit is an AWS-managed service. Our repositories are automatically encrypted at rest, also encrypted in transit. AWS handles that. And remember, when working with CodeCommit, we're making that choice up front how we want that handled, either HTTPS or SSH. Generally for Windows or Linux, it's just simpler to use HTTPS. If you're accessing it from a Mac, well SSH might be an option then. So CodeCommit's a managed service, data is encrypted in transit and in rest by AWS. Other parts of security, just think about IAM groups and attaching the appropriate policies to our groups. We can deny access to delete and create repositories. We can deny access to push to prod, or we can grant access to the appropriate personnel. So with that said, I'm going to end this lesson on CodeCommit Data Security. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeCommit Advanced

Hey, Cloud Gurus. Welcome to this lesson, CodeCommit Advanced. Now we've probably seen most of what is in this lesson, but this kind of ties it all together. And the last part of this lesson will go through what I'm calling a day in the life of a CodeCommit repository and just show all the things we can do, sort of chain them together in a real-world scenario. So with that said, let's look at some of the concepts, and first let's talk about migrating a Git repository to CodeCommit. So the scenario is we have a GitHub repo, and we want that in CodeCommit. We want to manage it over in CodeCommit, and we'll actually see this in our demo. So we have our local development machine. We have CodeCommit, and what we need to do first off or one of the very first things we need to do is create a CodeCommit repository. And we already have the GitHub repo, and we are going to clone that repo to our local dev environment. Now once we do that, and again, we already have a CodeCommit repository built, we can push our clone project to that new CodeCommit repo. And certainly from the CLI or from the Management Console, just go in and check and make sure that everything worked out okay. Now what about securing repositories and branches? So we have a use case. By default, any CodeCommit



repository user who has sufficient permissions to push code to the repository can contribute to any branch in that repository. So, if we create a branch, all of our users of that repository have access to that branch. Now this has caused problems. We need to limit pushes to prod to our admins. So we create a branch, but we don't want just anybody doing pushes to prod. So we have a dev team, and we have dev staging and prod repos, and we have admins. Dev team can push to dev. Well, that makes perfect sense. Dev team can push to staging. In most cases, that makes perfect sense as well. We don't want the dev team pushing to prod. And that's where policies are going to come into play, and we can create policies, which could limit the ability of our dev team to push to prod or completely eliminate that. We could attach a policy to a group in which our dev team are members of and just using common techniques that we've seen already. And from that point on, only our admin will be able to push to prod. All right, what about replicating repositories between regions? Well, think about other situations where we are going across regions. So there's going to have to be a trust created. We're talking about policies and roles. Let's see what this looks like. So we have a CodeCommit repository in Region 1. And how can we replicate this repository across regions? Well, just show a little extra here. Let's say we have a pipeline, CodeCommit, CodeBuild with a builds spec, and CodePipeline. And again, we want to replicate that repository across regions. It's easy to make this more complicated than it needs to be. We need to create a trust. This role is a service role, which allows us to work in Region 2. Now remember, I'm showing a pipeline. And by this point, we know all about triggers. Well, let's say we have a push to our repository, that triggers everything, and that could trigger this replication. The replication could be done with a lambda function. Again, most importantly, under the hood, so to speak, we have to have that cross-region permission set up. So we could have a trigger function, trigger this replication, or we could simply do a push over to the second region. Our demo, being mostly step 1, migrate a Git repository to CodeCommit. We'll get into the details of that in the demo. Once we get our CodeCommit repository set up, we can branch that repository, and we could take care when we branch on who has permissions to do what, merging our branch back in or pushing to prod. We can take care of that with an IAM policy. And then finally, set up a pipeline to replicate a repository across regions. And we can replicate with a trigger action, but we have to have those underlying permissions set up. So with all that said, let's go ahead and get into our demo and migrate a Git repository to CodeCommit. All right, out at the Management Console. So if we want to migrate a Git repository to CodeCommit, well, we need a CodeCommit repository, and I've already done this because we've seen this already. I created this demo repo. It's a Spring repository, aka Java, or we could do git create repository from the CLI. Our two CLI commands I'm going to use, CloudShell. And I know everything's here, but you could start issuing some of these commands. Just type in `aws`, `aws codecommit help`. And from that, we know we have the CLI, and CodeCommit's ready to go. Now from CloudShell, I don't need `aws configure`, but you could run that. Just remember you have to create your access key and secret access key and plug



those in. Don't need to do that here. Now for the user, you could go over to IAM and make sure that user has CodeCommit permissions such as CodeCommit Power User, and there we go. I'm logged in as Cael, so Cael is good to go with CodeCommit. Now remember to work with CodeCommit, let's scroll down. From the Security credentials tab, let's scroll down, HTTPS credentials for CodeCommit. So you would generate these credentials and download those credentials, and we'll use that momentarily, much like you would generate access keys and download those. All right, so we're all set up. What we need to do is clone that repository from GitHub, Spring repository, git clone, and then the URL that you can grab from the GitHub repository, the GitHub repository you're using. And locally, we want to name that aws-codecommit-demo. You just run that, and it's going to say it already exists. I've already done this. There's our new repo. So let's cd into that. And what do we want to do with this cloned repo? We want to push it to CodeCommit. It's just local right now. So let's go back over to our repository in the Management Console. Here's the repo I created. We need to grab this URL. Back over at the CLI, git push, our URL for our repo, and all, we want to push everything. Let's run that. What do you think will happen here? Think about it. Well, we'll get an error because I didn't have the all correct. Live and learn. Let's run that again. It wants those credentials, HTTPS, created that and then downloaded those. So you download them locally. We have to plug those in. So we have a username for Cael and a password. Plug that in. All right, it worked. It did not take long. And let's just verify. Let's go over to Management Console. Last modified 1 hour ago, update, Last modified Just now. So that worked. And what's in this repository is a bunch of code from a recent Spring Cloud AWS course that I did. Check that out if you're interested in Java. So that's a day in the life of an AWS repository. We could clone a repository from Git and push that to CodeCommit. And we can branch it then, and we could add permissions. Like in our example, only the admin can push to prod. And of course, we can replicate across regions either with the trigger action and a lambda or just do a git push across regions. The key point is we have to have that service role set up. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## CodeBuild

### Introduction to CodeBuild

Hey, Cloud Gurus. Welcome to this lesson, Introduction to CodeBuild. We're going to have three lessons on CodeBuild in this section. And again, this is an introductory, but we will certainly go deeper in the upcoming lessons. Now in this lesson, we're going to first just define what is CodeBuild? Once we do that, we'll talk about the different ways we can access CodeBuild. And an important part of CodeBuild is the output artifacts. Ultimately, that's what we're going to deploy



downstream. So we'll talk about the output artifacts a little bit, and then we'll wrap up with exam tips. So with that said, let's go ahead and answer the question, what is CodeBuild? So what is CodeBuild? It is a fully managed continuous integration service that compiles source code, runs tests, and produces software packages that are ready to deploy. So there's that key term again, fully managed. And we saw the benefits of CodeCommit being fully managed. We don't have to worry about encryption in transit and at rest. Well, with CodeBuild, there are many benefits as well. And most importantly, we don't have to maintain build servers. That is done for us by AWS. So let's focus on the output of CodeBuild for a minute. So CodeBuild produces software packages that are ready to deploy. And when I think of software packages, I've spent a lot of time working with Java, I think of a Java JAR file. We compile and build our code into a JAR file, and that's what CodeBuild is going to produce for us. Think of other types of code. What about just straight HTML, maybe front-end interface code or CloudFormation templates? And you're probably thinking, hey, wait a second. We don't need to build those, and you're right. So, in those cases, if that's the extent of our project, we don't need CodeBuild. And we'll see this when we get into CodePipeline, we have the option of choosing which of these services we use. Now by definition, a pipeline, it's going to need at least two services in there, maybe CodeCommit and CodeDeploy or CodeCommit and CodeBuild. But if we don't have anything to build, we don't have to use CodeBuild. But of course, if we have artifacts to build, well then we can choose CodeBuild. And just remember, this is crucial, it's an AWS-managed service. So that eliminates the need for us to create, manage, and scale those servers. All that headache is handled by AWS. And I suppose a good analogy to remember this is that think of Elastic Beanstalk. So we're handling our code, but recall, we don't have to do anything with the servers. So that's a really good way to remember this. CodeBuild is the same. We're dealing with our code, we're building our code, packaging it. We don't worry about servers. Now talking about scaling, I'll talk about billing here real quick, it's just important to have an overall understanding. So yes, CodeBuild scales to fit our needs. What we are charged for is when our builds are running. So it's not a charge based on hardware. It's a charge based on how many operations we're doing, how long our builds are running. So yes, CodeBuild scales up when needed. But when that need goes away, it's going to scale back down. So we don't have servers sitting there racking up charges on us. How do we access CodeBuild? Well, we can access CodeBuild from the Management Console, from the CLI, from the SDK. So programmatically, and I'll just use Java as an example, we can access CodeBuild within our Java code using the SDK and also CodePipeline. And we'll deal with CodePipeline in an upcoming section, ties all of this together. And with CodePipeline, when we're creating our pipeline, we can add CodeBuild as a build or a test action in our pipeline. And just remember, if we don't have anything to build such as just straight HTML or maybe JSON templates or YAML, well, we don't need a build stage. Now if we decide to use CodeBuild, remember we're working with code, and we're going to have artifacts from our build. Where do we keep these



artifacts? Well, we're going to keep them in S3. So initially, we grab the code from our repository. I'm showing CodeCommit. We could use other repositories. There's our CodeBuild. Key points, CodeBuild is fully managed, no need to provision and scale servers. And it can provide pre-packaged build environments for many common programming languages. We perform our build. We need to store those output artifacts. We store them in S3. Now we're storing artifacts. We want to turn on versioning for S3. We want encryption at rest. We can begin to see a pipeline forming here. Here's our source code. We perform a build, and we output the artifact into S3. Where do you think this goes next? Well, good answer would be CodeDeploy works with our build, and we'll certainly cover CodeDeploy in the next section. So you see the makings of our pipeline here. All right, so I wanted to provide a brief introduction to CodeBuild. We'll go much deeper in the next two lessons. Let's just review some tips. How can CodeBuild be used? It can be added to the build stage and also the test stage in a pipeline. And these pipelines are sort of sequential. Bear with me. We will certainly cover the rest of our pipeline, CodeDeploy and CodePipeline, in the next two sections. You must provide CodeBuild with a build project, and we'll get into that in the next lesson. What is a build project? This is sort of an introduction to the next lesson. So basically, the build project is going to tell CodeBuild how to run a build, including where to get the source code, which build environment to use, which build commands to run, and where to store that build output. So I wanted to touch on the build project in this lesson. Next two lessons, we'll know much more about it. But remember our source, CodeCommit or another repository. Remember our output. Where are we storing our build artifacts? We're storing them in S3, source, build, build output. Now one more exam tip, I want to do something just a little bit different. But I think it's a great learning tool, especially if you're new to these services, CodeCommit, CodeBuild, CodeDeploy, CodePipeline. Let's go out to the Management Console. All right, so I'm at the Management Console, and we're not going to create a project, but I just wanted to show, and this is certainly an exam tip, especially, again, if you're not familiar with these services or not very familiar. All right, and remember our diagram. The source, what are potential sources for us? There we go, S3, CodeCommit, GitHub, Bitbucket, GitHub Enterprise. And just remember with S3, for it to be a repository, we have to have versioning on. The point is, in the Management Console in these services, study the drop-downs. And there's your answer for what can be our source for CodeBuild. Where do we store our artifacts? S3. So just studying those drop-downs, it sort of ties our diagram together, CodeCommit, CodeBuild. I'll put artifacts in S3. And certainly, when we get to CodePipeline, there's the whole picture. And studying those drop-downs will be a great tool for preparing for the exam. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeBuild Walkthrough - Demo



Hello, Cloud Gurus. Welcome to this lesson titled CodeBuild Walkthrough, and we're going to do just that. We're going to walk through the steps that it takes to perform a build in CodeBuild. We will look at some use cases and do a bit of review and exam tips at the end of the lesson. So let's take a look at the steps and how CodeBuild works. The first thing we need to do is provide CodeBuild with a build project, and our build project is going to tell CodeBuild how to perform the build. It's going to tell CodeBuild where to get the source, what environment to use, what commands to use, and where to store the build output. And we talked about that in the last lesson. We know where to store it. We're storing it in S3. Step two, CodeBuild uses the build project to create the build environment. Now step three, there's our source code. So it downloads it into the build environment and uses the build specification, otherwise known as the buildspec file. And that buildspec is a collection of build commands and related settings that CodeBuild uses to run a build. And we'll check out a buildspec in the next lesson. Step four, CodeBuild sends the output of a build to our output repository S3. Other things it can do is set up notifications. So we could have SNS involved here as well. Step five, notice we have two things going on here. While our build is running, our build project can send information to CloudWatch. Information can be sent back to CodeBuild. And also, while our build is running, we can use our interface of choice, Management Console, CLI, SDK, CodePipeline to get updates on our build. And just notice where we're going to get that information, CloudWatch Logs. We can be looking at CodeBuild from all of these interfaces. We'll see when we get into the CodePipeline section and look at pipelines, we'll see all of this going on. All the way from CodeCommit, if we have a build stage, CodeBuild, CodeDeploy, we'll see that all in CodePipeline. So if we were going to perform this build, and again, I'll refer you to that supplementary lesson if you want to see that in action or actually walk through it yourself, first thing we need to do is work with S3. We need an input bucket, and we need an output bucket. Where's our input coming from? Our input is coming from CodeCommit. We output it to S3. That output will be the input for the next stage, CodeDeploy, or the deployment stage to be more specific. We can certainly use CodeDeploy. We could also use Jenkins with AWS plugin for our deployments. And just a couple of key points about our buckets. Remember when we're doing something like this or working with CodeCommit, we want these buckets versioned. Additionally, it's a requirement that these buckets are in the same region as our build. And I want to really emphasize the inputs and the outputs. Just understand this, so our input is our source code, input to CodeBuild. That source code is in a sense the output from our source stage. I was going to say CodeCommit, but it could be any repository. So that output, the source code from CodeCommit, becomes the input for CodeBuild. The output of our build project, and I'll stick with Java, it would be a JAR file. We store it in S3. Well, guess what? The next stage, which we'll cover in the next section, CodeDeploy has access to this S3 bucket. That JAR file becomes the input to CodeDeploy. And you can see we're starting to chain this together, the makings of a pipeline. Other key components of this process are the build project, the buildspec file. Again, we'll see this buildspec in the next



lesson. And it's basically a roadmap on completing this build. So just remember those key components are build project and a buildspec file, and we ultimately create a build environment. Now just remember, as input, we must provide CodeBuild with a build project. We could create that from the command line or from the Management Console. And I'll refer you one more time to that supplementary lesson, and we do just that. So just one more overview. We need to create a build project. We need to provide the source code and also a buildspec file, which basically gives CodeBuild directions on performing the build, where to store everything. Our input is coming from S3 or CodeCommit. Our output is S3, and our output artifact, I'll use Java as an example, a JAR file, that will ultimately be the input to the next stage, CodeDeploy. All right, so let's talk about a few use cases just to add on to this a little bit. So let's say that our development team is working, they're branching their code, and then they merge that code back into CodeCommit. And we want to run CodeBuild after a successful merge to CodeCommit. How do we do that? We need automation. So what's our automation tool? We get CloudWatch Events involved. So we set up CloudWatch Events to monitor our repository. And when we get a successful merge, that merge will be a trigger for our CloudWatch event. CloudWatch event can trigger CodeBuild to run a code on that just merged source code. So that's one use case where we're thinking about automation. Again, that's a key emphasis in this course. And how do we automate? Well, CloudWatch Events. Other ways to automate, Lambda. We could have a lambda function set to trigger on an update to CodeCommit or merge. But in this case, CloudWatch Events will do just fine. There we go. That's our solution. All right, let's look at another use case. We need to review CodeBuild logs and notify personnel of new logs. So what are we doing here? So, first thing we want to do is send our CodeBuild logs to CloudWatch Logs. We can again set up a CloudWatch event. Again, when we're talking about automation, we really want to be thinking about CloudWatch Events. And just remember, CloudWatch Events has been renamed to EventBridge Events. It's the same API, and it's just a timing situation. I sort of have to mention both because it's unlikely that the exam will be consistent at this point. It's fairly new. You may see CloudWatch Events, you may see EventBridge Events. Just remember, it's the same thing, same API. EventBridge Events are CloudWatch Events with some extra functionality. But either way, we're monitoring for CloudWatch Logs. And here's another one of our key automation tools, we can trigger a lambda function to send a notification to Slack. Not showing in the diagram, we could just set up an SNS topic if we just wanted to notify someone via email. All right, let's review a couple things, and these are really key takeaways. What are our inputs? You must provide CodeBuild with a build project. You're not going to go anywhere without that build project. You can create that using the CLI. You can create that in the Management Console outputs. We are outputting to S3. I'll stick with the Java example. The output would be our Java JAR file. That output becomes the input to our next stage. And of course, going back to inputs, obviously we need the source code as well, either coming from CodeCommit or S3 bucket with versioning turned on. Let's cover a



couple exam tips. We talked about this more in the last lesson than this lesson. Do we need a build stage? Only if our code artifacts need to be built and packaged, going back to the Java versus simple HTML example. Simple HTML, we're not building anything, so we don't need a build stage. Java, yeah. We're building a JAR file. We need a build stage. Just remember for our pipeline, this stage is optional. Of course, depending on hey, are we building anything or not building anything? And what is a buildspec? Well, we'll get into that deeper in the next lesson, but a takeaway is it is a collection of build commands and related settings in YAML format that CodeBuild uses to run a build. So it's sort of the conductor. It tells CodeBuild exactly what it needs to do, where to put outputs, and what commands to run. So that's a look at the key points of how we would perform a build with CodeBuild. And one more time, if you want to go further and actually perform a build, check out the supplementary lesson in the lesson materials for this lesson. And I walked through all of that using the command line and a little bit of the Management Console. And with that said, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## Working with Build Projects and the Buildspec File

Hey, Cloud Gurus. Welcome to this lesson, Working with Build Projects and the Buildspec File. So we've already sort of walked through a build. We talked a lot about the buildspec file. But in this lesson, we're going to go really deep on the buildspec file. And additionally, we'll take a look at the steps performed in a build, and they sort of relate to the buildspec file. We'll see momentarily that the buildspec file has steps and stages as well. So really, the heart of a build is the buildspec file. And maybe even more appropriately, the brains of a build is the buildspec file. And we can really orchestrate builds using the buildspec file. Basically, it tells our build exactly what needs to be done, where everything needs to go. So looking at our diagram, we have our source code. We need to create a build project. Build project creates a build environment, but we need a buildspec file. So let's take a look at the steps in a build project. So our project is submitted, and then provisioning starts. Provisioning what? Well remember, CodeBuild is a managed service. So we're not thinking about servers, but we need build servers. Well, AWS is handling that for us. What are we going to build? Well, we need our source code. Typically, our buildspec file is zipped up with the source code. So we need access to our buildspec file. There's a pre-build phase, and I'll get into more detail on that when we look at a buildspec file, which again is in a YAML format. Then there's the build phase. Then there is post-build. What is coming out of that? Artifacts. I'll just use Java again as an example. From Java, a JAR file is produced, and that is stored in that output S3 bucket. Then there's some finalizing going on, some clean up and completing. And let's go back up here. We have a few more things going on. Just to recap though, ultimately our build output artifact is going to S3. Let's say it's a JAR file. Sneak preview, what's the next step in our stage? Well, we're probably going to



deploy, and this will make a little bit more sense when we look at a build file momentarily. All right, so let's talk about our buildspec file. What is a buildspec? It's a collection of build commands in related settings, key point in the YAML format, that CodeBuild uses to run a build. How does buildspec work with the build environment? You can include a buildspec as part of the source code, or you can define a buildspec when you create a build project. Well, it's sort of required. That's a benefit of having one. You can run a build. But again, as we'll see momentarily, the whole brains of the operation is the buildspec. What commands are we running? Where do we put the output? All right, let's check out a couple more points here. If you include a buildspec as part of the source code, by default, the buildspec file must be named buildspec.yml and placed in the root of your source directory. Now if you think about that, so, this file is bundled with your source code. It sort of simplifies things if it's named buildspec. So CodeBuild sort of knows what to look for and where to look for it in the root of your source directory. You can specify only one buildspec for a build project regardless of the buildspec file's name. So, remember if you bundle it with a source code, you have to name it buildspec. If it's not bundled, you could name it something else, but there can only be one. So with all that said, I'm going to jump out to an editor, and let's take a look at a buildspec file. All right, so I named this file buildspecstub because it's sort of a skeleton of a buildspec. I use this when I want to create a buildspec. This is a good starting point. All right, so we have this version number. Surprisingly enough, version 0.2 is recommended. All right, then we have run-as, and this is optional. It's for Linux users only. And as its name implies, it specifies a Linux user that runs commands in the buildspec file. Environment is optional. But if we use it, we can set some environment variables such as our ACCESS\_KEY, SECRET\_ACCESS\_KEY, DEFAULT\_REGION, PASSWORD. Now the question is should we store an access key and a secret access key in this file? It's kind of like bungee jumping. Even though you can, maybe you shouldn't. So it's really not good practice to store such things in files. We have things such as parameter-store and secrets-manager where we can store such things. So disregard ACCESS\_KEY and SECRET\_ACCESS\_KEY being in here. I put it in here just to emphasize a point. But that's where you would put environment variables. And great segue. Notice just below, parameter-store. We can specify variables to reference those secret things that we store in parameter-store, git-credential-helper. It's either a yes or a no, is CodeBuild using git-credential-helper to provide credentials? Proxy is also optional. And it's used if you run your build in a proxy server. Okay, now we're really getting to the heart of the matter. We're down to phases. Phases are required. So this phases section is required, but not all of its contents are required. And remember, we step through the diagram, install, pre-build, build, post-build, artifacts. There's our run-as. Same thing, this is for Linux users. Then we have the commands that CodeBuild will run during installation, and install is specifically for installing packages. A good example would be to install a code testing framework such as Mocha. We have pre-build. Again, this is optional, Linux users. Then our commands for pre-build, and then a finally, which is cleanup. Think of, if you're



into code, think of exception handling. Is there something that went wrong? Is there something we need to fix, something we need to be aware of? A good example of pre-build commands would be, let's say we need to sign into ECR. We would do it in pre-build. Get down to build, of course, these are the builds CodeBuild would run during a build. Then we have post-build. An example of this would be Maven, Apache Maven we could use to create our JAR file. A note on these finallys. They will run no matter the outcome of the command. Then we have artifacts, files, location. Where's our code? Where do we put our output artifact? And as we know, our output artifact will go in an S3 bucket. All right, so let's take a look at an actual buildspec file. So this one's a stub and pretty much contains a skeleton of everything we could possibly want to do in here. Typically, a real buildspec is going to be smaller. And notice, no environment variables. We do have the version. Phases is mandatory. What we do in there, we have a little bit more flexibility. Install. All right, just a comment. Nothing to do in the install phase, nothing to do in the pre-build phase. Build. Well, we're using Apache Maven to perform an install. Post-build, nothing really going on with the exception of just a little bit of communication here. Artifacts. That artifact is our output artifact that we're storing in S3. And in this case, messageUtil.jar. Now, as sort of just a working example, what if we needed to install Java runtime? We could do something like that. And I just did that to emphasize the flexibility here. Again, phases is mandatory. What we're doing inside of phases, all these stages and having actual commands in there is optional. Notice that artifacts is at the same level as phases. So it's not actually part of phases. We do need to tell, hey, CodeBuild, that's our artifact. That needs to end up in S3. So a good takeaway from the buildspec file is, of course, phases is required. Know what we can do in each of these phases, but they're not required. We just do whatever we need to do. Hey, if we need a runtime, sure, we can do that during installation. Are we doing a build? What are we building with? How about Apache Maven? That will work. And then finally, artifacts. Output artifact for Java, it would be a .jar file, and we store that in S3. All right, just to touch on our steps in a build project real quick, and notice some of these steps are some of our phases, install, pre-build, build, post-build. Ultimately, we want to upload our artifacts. Where? S3. Output from CodeBuild becomes input to our deploy stage. So we looked at the inner workings of a buildspec file. And just remember, we need that buildspec file. Buildspec file is pretty much telling CodeBuild the commands such as hey, are we installing? Yes, we're installing with Apache Maven. And what's the name of the JAR file? The JAR file needs stored in our output artifact. Here are some questions to ask when planning a build. Where's the source code? Where's our repository? Is it S3? Is it CodeCommit? What's its location? Which build commands and in what order? We might sketch these out on paper and then build the buildspec file using these commands, inputting these commands where appropriate in the phases of our buildspec file. Which runtime and tools are needed? If we need a runtime such as Java, we can add that into our install phase. Our example showed Maven. In the build phase, Maven, Apache Maven, is actually doing the build. Here's one I haven't mentioned in this lesson, but it's



always a consideration. Think about CloudFormation, how many services CloudFormation touches. Are we touching other services here? Is a service role needed to work with other AWS services? That is always a question. Whether it's CodeBuild or are we doing something with Lambda or CloudFormation, always ask that question. Is CodeBuild working with your VPC? Well, what if we're doing a build and we're working with EC2 by definition, unless we're going really way back, EC2 is going to be in a VPC. So what do we need? A VPC ID. What else do we need? How about security groups? How about one more ID? Think about that for a second. Subnet IDs. So that's a look at the buildspec file and how this buildspec file orchestrates our builds in CodeBuild. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeBuild Advanced

Hey, Cloud Gurus. Welcome to this lesson, CodeBuild Advanced. An alternate name could be CodeBuild extras. Just don't be afraid of the word advanced. Nothing exceptionally difficult in this lesson. So let's get started. Have a little conversation here. I've got this complex buildspec.yml file. I wish I could test it locally. You can do that. Do you mean that LocalStack stuff? Well, that would work, but it's overkill. LocalStack is for running a cloud on your local machine and entails most AWS services. So what do I do? You can run builds locally using the CodeBuild agent. So basically, you can set this up on your local machine, set up the build image the first time you run the CodeBuild agent, run the CodeBuild agent locally. And yes, you can perform tests. And certainly a good use case, test out your buildspec.yml file locally. Just something to be aware of. What are some benefits? Again, you can test buildspec.yml before actually using it in CodeBuild projects. Test and build applications locally before committing changes, and identify, fix, and experiment with things quickly. All right, so we have local dev, CodeBuild. We install Git on our local machine. Install and set up Docker locally. Set up the build image. You only need to set up the build image the first time you run CodeBuild agent. Where do you get this build image? You can pull it from CodeBuild public Amazon ECR repository. Download the CodeBuild agent, and run the agent. Start testing locally. Pretty cool stuff. All right, CodeBuild with VPC. What is this? Typically, CodeBuild can't access resources in a VPC, but we can enable this. Let's talk about some use cases. Run integration tests from your build against data in an RDS database that's isolated in a private subnet, of course, within your VPC. Query data in an ElastiCache cluster directly from tests. Interact with internal web services hosted on EC2, ECS or services that use internal load balancing. All right, that sounds helpful. Let's check out a demo and set up VPC access for CodeBuild. All right, I'm out at CodeBuild in the Management Console. Let's go ahead and Create project. Give our project a name. Source provider, we'll use CodeCommit. And from that, I selected DemoRepoSpring. Branch, main. I'm just going to accept defaults here. And we scroll down. Additional configuration note in here is VPC,



and I select a VPC that I created. Now let's read the fine print here, select the VPC subnets that CodeBuild should use. For high availability, we recommend selecting multiple subnets from multiple availability zones. Well, this is just a quick demo. I'm not going to do that. But what we must do, ensure your subnets include a NAT gateway. Now, a NAT gateway implies a private subnet. I just have a couple tabs open, and I created a NAT gateway for this subnet. And a key point here, I'll click on that subnet. Let's go to Route table. Notice there's no route involving an internet gateway. So this is a private subnet with a NAT gateway. Note the subnet ID. I'll select that. I'll select a security group. Let's validate again. Let's validate. CodeBuild can't find the destination for the target NAT gateway. All right, we can rectify that. So we go into our route table, we go to Routes, Edit routes, Add route, NAT Gateway. There's our gateway we created. Let's save changes, and we need a CIDR, Save changes. Okay, so we have that route to the NAT. Let's go back, and let's create our build project. All right, so now our build project can access resources within that VPC. All right, let's just talk about environment variables. And AWS CodeBuild provides several environment variables that you can use in your build commands. And this is not all inclusive. It provides `DEFAULT_REGION`, `REGION`. Note the difference for CLI and for SDK. `BATCH_BUILD_IDENTIFIER`, the `BUILD_ARN`, and we can store our own environment variables. So this sounds awesome! Can we store the access key in these variables? We know better than that, correct? Let's be smart. Store access keys in our secure managed services such as Parameter Store and Secrets Manager. Even better, maybe there's an alternative to using access keys. All right, last, but not least, let's talk about automated code review on pull requests. So a developer raises a pull request against the main branch of our source repository. The pull request event is received by the default event bus, and we're talking about EventBridge in this case. The default event bus triggers the EventBridge rule. The EventBridge rule triggers the CodeBuild project. So then, the AWS CodeBuild project runs the code quality check and sends the results back to the pull request as comments, and this pull request can be approved or rejected automatically. Just more important to understand how this works. It wouldn't be a huge feature on the exam, but certainly give us a good understanding of some of these automated processes. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## CodeDeploy

### Introduction to CodeDeploy

Hello, Cloud Gurus. Welcome to the lesson on Introduction to CodeDeploy. So this will be a quick introductory lesson, but we have a few more lessons where we'll go much deeper on CodeDeploy. So what is CodeDeploy? Well, CodeDeploy



is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, so we can go the whole way across from AWS to on-premises instances, serverless lambda functions or Amazon ECS services. So before we go much further, let's talk about what we can deploy. Well, the obvious first answer is, of course, we can deploy code. We can deploy Java code or C#. We can also deploy lambda functions, whatever code they happen to be written in. We can deploy web and configuration files, executables, packages, scripts. We can even deploy multimedia files. Now what's our key term for the course? We want to automate this, and realize that these deployments may be to dev or test or prod. And our goal is to automate our deployments. So let's reveal our diagram a little bit here, and this is a little bit tricky intentionally. So where is our code coming from? Well, most likely our code is coming from S3. We can also use Bitbucket or GitHub. And you're surely thinking, wait a second. What about CodeCommit? And again, this diagram, I made it a little bit tricky just to get you thinking about it a little bit. Well, of course, our code can come from CodeCommit. But the diagram, the emphasis is on remember our inputs and our outputs. Remember where CodeCommit is outputting a package. Ultimately, if we run a build, CodeBuild is grabbing the code from CodeCommit, performing the build, and usually placing that build in S3. Again, there are other services that can be used. So remember our pipeline, and this is sort of a snapshot of that pipeline. So, our output from CodeBuild, S3, is input to CodeDeploy. Yes, absolutely CodeCommit can be in this pipeline. It's just further back to the left, so to speak, in the pipeline. Now what about where can we deploy to? Well, I'm showing an EC2 placement group. We could deploy to a single EC2 instance. We could deploy to an Auto Scaling group. Large organizations, we may be deploying to hundreds of EC2 instances at once. Can we deploy across regions? Yes, we can deploy across regions. So think about that. When I said hundreds of EC2 instances, those EC2 instances could be dispersed across regions. Now we've already talked about rolling deployments, and in a large scale, we certainly can do rolling deployments and reduce our downtime. All right, so that's an overview look at CodeDeploy. It's pretty straightforward, and we'll certainly go deeper in the upcoming lessons. Let's talk about setting up and configuring CodeDeploy. And as is often the case with AWS services, a good place to start is with IAM. So, we would want to provision a user with access to CodeDeploy. So we'd attach a CodeDeploy policy to the user. Again, good IAM practices. If we have multiple users, we can certainly create a group and attach a CodeDeploy policy to that group. So much of this configuration for AWS services is very, very similar. Think about what the next step may be here. Ah, we create an instance profile to allow EC2 to access our repositories. So again, I'll reference back to CodeCommit. Code is in CodeCommit. CodeBuild grabs that code, performs a build. Remember our output artifact. Our build package is placed in the S3 bucket. So, when we say we create an instance profile to allow EC2 to access our repositories, we want to allow EC2 to access that S3 output artifact bucket. And that's why I made the diagram above a little bit tricky because I don't know about anyone else, but my mind is thinking CodeCommit in this situation. Well, we have



a build, we have a package, we have an output artifact. Where's that at? It's actually in S3. That's what our instance profile needs to give access to. All right, next step, a little bit more access here. We create an IAM service role that will grant CodeDeploy access to other resources. And that makes sense. CodeDeploy wants to deploy applications on our EC2 instances, our Auto Scaling groups or even lambda functions. Well, whatever resources we're dealing with, that service role needs to grant CodeDeploy access to those resources. So let's think about this a little bit and plan this. How would we do this? Well, remember, CodeDeploy can deploy to EC2, lambda functions, and ECS. So why don't we think ahead? And if we're in a large organization, the very real possibility exists that we would ultimately be deploying to all of those platforms. So the way to do this is to create a service role for each of these platforms, and then we can just reuse that service role in the appropriate situation. So remember this three-step process, provision an IAM user or a group of users with access to CodeDeploy. So we attach a CodeDeploy policy to the user or a group of users. Create an IAM instance profile to allow EC2 to access your repositories, and just don't get tripped up on that. Remember, our output artifacts are in S3. And create an IAM service role that will grant CodeDeploy access to other AWS resources. And if you think you're going to be doing EC2 deployments, create a service role for that. Lambda, create a service role for that. And create a service role for ECS. Not mandatory, but if you deploy to all three of those, go ahead and create those service roles for each. Those are the key points to remember about configuration. All right, exam tips. And I just wanted to make a big emphasis. This can get a little bit tricky. The most common repository for CodeDeploy artifacts. Little bit of a word game here. Just remember where our CodeBuild output artifacts are going. They're going to S3. That's where CodeDeploy is going to pull that build package. I always use Java, a JAR file example. So in this lesson, we did an introductory look at CodeDeploy, certainly where CodeDeploy is getting the package from. Of course, it's S3. Where it can deploy to, EC2. And think Auto Scaling groups, placement groups, anything associated with EC2, lambda functions, and ECS. You can also go across and do on-premises deployments. So that's it. I'm going to end this lesson. We'll go much deeper on CodeDeploy on upcoming lessons. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeDeploy Setup and Configuration

Hey, Cloud Gurus. Welcome back. And in this lesson, we're going to talk about Code Deploy Setup and Configuration. Now in the previous lesson, we did a sneak preview. And in movie terms, they call those trailers. We checked out a trailer for setup and configuration. But certainly part of setup and configuration for CodeDeploy is getting those IAM pieces into place. So we talked about a CodeDeploy policy for users, and we could certainly attach that to a group of users. And we also talked about creating an instance profile and a service role for



CodeDeploy access. Specifically, it's going to grant access of CodeDeploy to other services. Remember where we're deploying to EC2, maybe a lambda function or ECS. And if you anticipate dealing with all of these services, you can create a service role for each of those and then just reuse them. All right, so with the IAM and permissions handled, let's talk a little bit about deploying, what we're going to deploy, how we're going to deploy it and at what rate and where to deploy. So the term for what we are going to deploy is revisions, and we can deploy multiple revisions. So I'll just stick with the Java example I've been using. And we have a Java application, and we have some updates to that application. We build and package those updates into a JAR file. That is a revision. That's what we want to deploy. What about how to deploy? Well, we need to set up a deployment configuration. And really, our deployment configuration answers that question, how to deploy and at what rate. And when you think about it, the speed which we execute these deployments can be an important factor and a consideration. Is downtime acceptable? That's a big key question. And what about disaster recovery? What about disaster recovery drills where everything is time-based and we need to know how long these deployments are going to take? And then finally, where do we deploy? We deploy to a deployment group. Now don't be misled by that. That deployment group could be a single instance, it could be a group of instances, an Auto Scaling group. And for EC2, we can deploy to Amazon Linux instances, Ubuntu, Red Hat. We can deploy to Windows Server. And just remember, I'm focusing on EC2, but we can deploy to Lambda and also ECS. Now can we install to just any EC2 instance? Well, yes and no. We can certainly install to an EC2 instance. That instance needs to have a CodeDeploy agent installed. And the typical way to install that is when you spin the instance up in the user data in the script, you can install the CodeDeploy agent on that instance. Now I'll provide a link in this lesson. And for those of you who want to get hands-on, we have a lesson to do just that where we're working in the Management Console, and we configure an agent for deployment and deploy to it. And key takeaways from this diagram, again, what to deploy. Well, it's going to be our build package such as a Java JAR file known as a revision. How to deploy and at what rate? Our deployment configuration is going to specify that. Where to deploy, deployment group, could be an EC2 instance, could be an Auto Scaling group, could be a lambda function or Elastic Container Service, ECS. Now taking this a step further, we want to add some automation. It's really a big focus on AWS DevOps. All right, so we have CodeDeploy, and there's a lot going on in these deployments. And when we're running the deploy, we may want to keep key administrators apprised of what's going on. Certainly, a person or small group of persons will be executing the deployment. But hey, what about automation? This deployment may be kicking off and it's not hands-on. So how do we notify folks? How do we automate this? Well, remember AWS EventBridge, pretty much took over for CloudWatch Events, some extra bells and whistles. So with EventBridge, we can monitor our deployments. And we could create an SNS topic and subscribe to that topic. So our EventBridge is monitoring specific CodeDeploy events. We perform a deploy, triggers our automation. EventBridge



sends information to SNS. We're subscribed to that topic. We're notified via email. What if it's more urgent? And in a lot of cases, it would be more urgent to know what's going on with our deployments. Ah, we can trigger a lambda function, and that lambda function can be set up to place a notification in a Slack channel. So that is almost immediate. So when we talk about automation, and a lot of times the end product of our automation is some sort of message, well think about what that's doing for us. If we're being notified almost immediately via Slack, well, then we can be off doing other things. We don't have to be focused on what we want to be automated anyway, our deployment. It's certainly a very big deal, but at the very least, we can be multitasking and apprised in real time. And if something happens, something goes wrong, well we can certainly deal with that. All right, just to reinforce the idea, what can we deploy? We can deploy scripts. Of course, we can deploy code, serverless lambda functions, executable packages, web and configuration files, and even media files. Now remember the difference between some of these just to reinforce another point. Going back to that question, do we need a build stage? Well, for code, we need to build that code. Yes, we need a build stage. For scripts, most likely we don't need to build those scripts, so we would not need a build stage here. In the next group of lessons on CodePipeline, we'll see graphically our pipeline, but the difference is here maybe we have CodeCommit, CodeBuild, CodeDeploy. In this case, we have CodeCommit. We don't need a build, so we don't need a build stage. CodeCommit goes right to CodeDeploy. So it's important to start tying this together and actually seeing the pipeline, and we'll certainly see that when we get into CodePipeline. All right, how do we deploy? Well, CodeDeploy can deploy application content that runs on a server, and it's generally stored in S3 buckets. You can also use GitHub repositories or Bitbucket. You can also deploy a serverless lambda function. Just remember that tricky concept, thinking, hey, what about CodeCommit? Well remember, CodeBuild grabs that source from CodeCommit or whatever repository performs the build and outputs it to S3. Our output artifacts are in S3. That's where CodeDeploy is going to be looking for those artifacts, in an S3 bucket typically. And the deployment method we choose will determine the rate of deployment. We'll talk about that momentarily. Where to deploy, again, EC2, on-premises. We can deploy to Lambda. We can deploy to ECS. Key points about EC2. It could be a single EC2 instance. It could be an Auto Scaling group. It could be a placement group. Most importantly, remember that that instance or instances needs the CodeDeploy agent installed on it. We would do that usually when we're spinning up the instance in the user data. So in this lesson, we've talked about what to deploy, certainly can deploy code, packaged code such as Java JAR files. We can deploy scripts, know the difference between the two. And again, trying to tie this all back together, do we need a build stage? How to deploy our deployment configuration is going to basically manage how we deploy and at what rate. Exam tips, other than those key points, just remember, automation is just a big factor. Remember our automation tools. EventBridge can be watching our CodeDeploy operations and trigger SNS or



Lambda. Create a topic, subscribe to the topic, or it can just trigger Lambda. What's the difference? SNS we can send to email. Lambda can notify Slack and give us virtually a real-time update. With that said, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeDeploy Application, Deployment Groups, and Deployment Configurations

Hey, Cloud Gurus. Welcome. In this lesson, we're going to learn about CodeDeploy applications, deployment groups, and deployment configurations, and these are the key elements in deploying our applications. So let's first talk about these key elements, and we'll start with an application. And we all know what an application is, right? Well, we do, except for CodeDeploy, it's not quite what we're used to. So what is an application in CodeDeploy? It's simply an identifier used to reference your deployment settings. I know it's really strange, doesn't make a lot of sense, but just remember what an application is in the context of CodeDeploy. So it's not a software application. It's pretty much just a name identifier, references our deployment settings. All right, so what is a deployment group? It is the instance or instances where you want to target and deploy your code. And it's another one on a little bit shaky ground because even if you're deploying to one instance, that one instance will be your deployment group. So just really know these terms for CodeDeploy because it is a little bit tricky. And can our deployment group be made up of any instance or instances? Well, remember from the previous lesson, key point, it needs to have the CodeDeploy agent installed on it, and remember where that happened. When we spin up the instance, in the user data, we can install that CodeDeploy agent. All right, what is the deployment configuration? A deployment configuration is a set of rules and success and failure conditions used by CodeDeploy during a deployment. So the rules are different in the conditions, depending on where you deploy. Are you deploying EC2, on-prem, Lambda or ECS? And we'll take a look at that later in the lesson. Let's focus a little bit on EC2 and on-prem. So, when we deploy to EC2 or on-prem, our deployment configuration is going to specify the number or percentage of instances that must remain available during the deployment. And it does that using a `minimumHealthyHosts` value. So there's sort of the official definition. One way I like to think of it is it's the number of instances for which you want to deploy your code. Certainly remember the official definition. This is very simplified, but it sort of cuts to the chase. So, when we do a deploy, we have a workflow. Let's take a look at that workflow. So we first create the application, and again, that is a CodeDeploy application, not how we think of an application. Think in terms of CodeDeploy. And then we create a deployment group. And just a little bit of trickiness here, even if it is one instance, it is still a deployment group. We then specify a deployment configuration. So we specify how many instances we're deploying to at one time, and we use the variable `minimumHealthyHosts` in



that situation. We then upload our revision. Well, that's another CodeDeploy term. That is the package we want to deploy. Maybe a JAR file in Java is a good example. With all that specified, we can go ahead and deploy then. Then we can go ahead and check the results. Now, if our results are unexpected and we're not happy with those results, we can go back, make the necessary changes, and deploy again. And yes, we can automate our rollbacks as well. And item number 7 really sums it up. We redeploy as needed. So for our deployment configuration, how do we want to deploy, how fast? We can do one at a time, all at once or half at a time, and we'll talk about some of the drawbacks here. But think about it. If we're doing one at a time, we're sort of hedging our bets, and it's safe. If something goes wrong, we only have one instance that has changed. Unless, of course, we only have one instance that we're deploying to. All at once, well we're really confident, let's just go ahead and do it, or hey, it's not a critical deployment, let's just do it all at once. Just realize if you're doing an all-at-once deployment, you may have a little bit of downtime, and you may have a messy rollback as well. And again, with half at a time, we're sort of hedging our bets a little bit, providing some protection in case something goes wrong. And then the flip side of that is speed of deployment. Well, makes sense that all at once is going to be a faster deployment than one at a time. So the level of risk you're willing to accept, as well as the speed of deployment, they're converse to each other, but they're considerations that we have to think of. All right, so our EC2 deployment group, set of individual instances targeted for deployment. And again, even if it's just one, it's usually multiple, but if it's just one, that's a deployment group. Now what I want to do, we need to understand the options available for each type of deployment group. And I'm just going to go out to one of the AWS pages on this topic. So this is a good page to bookmark and maybe skim through when you have the time. It's talking about deployment configurations, in this case, configurations on EC2 or on-prem. And here it's talking about the all at once, two different kinds in-place deployments, blue/green deployments. So I'm not going to read through all of this, but just wanted to point out this is a really good page to summarize our deployment configurations. And I'll scroll down a little bit. Here's some information on half at a time and then one at a time. It also covers our other deployment options such as ECS and Lambda. It's a good page to skim in exam prep and even more so on the job. This will come in handy for doing deployments. So that's a look at CodeDeploy applications, deployment groups, and deployment configurations. Just remember the term application, specifically to CodeDeploy is not as we think of an application. Deployment groups, again, even if it's just one instance we're deploying to, that's still our deployment group. And then deployment configurations. How quickly are we deploying, one at a time, half at a time, all at once? And this brings into play our risk, tolerance, and also how quickly we want to deploy. So with that said, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.



## CodeDeploy appspec.yml

Hey, Cloud Gurus. In this lesson, we're going to talk about the CodeDeploy appspec file. Now if you remember from CodeBuild, CodeBuild had a buildspec file, and it was sort of the brains of the operation. The concept is similar here. The appspec file, and this is unique to CodeDeploy, is a YAML or JSON-formatted file, same as CloudFormation templates. And it is used to manage each deployment as a series of lifecycle event hooks. Where have we heard that before? Well, auto scaling has lifecycle event hooks, pretty similar concept. And these event hooks are defined in the file, and we'll take a look at a file momentarily. All right, let's take an overview look at the appspec file. So when we want to deploy, we have our deployable content, whether it's a Java JAR file or a bunch of scripts or whatever we want to deploy. And then, the appspec file is sort of a manifest of how we're going to do this deploy, where we're getting files from, where we're putting files, what we're going to do before installation, after installation, what to do at application startup. And again, we'll get into those details momentarily when we look at an appspec file. So of course, CodeDeploy grabs the deployable content, grabs the appspec file, and performs the deployment. Where are we deploying to? Remember our deployment group. It could be one EC2 instance, an Auto Scaling group of instances. It could be Lambda, it could be ECS, but they all utilize this appspec file. So there's a lot going on with our appspec file. But to summarize it in one sentence, what does an appspec file do for us? It manages our deployments, and it will become very crystal clear when we see the code for the appspec file. Again, what is the appspec file? Well, it manages our deployments, it's unique to CodeDeploy, and it's in YAML or JSON format. We'll certainly be familiar with that. That's CloudFormation templates as well. File structure. So, we're going to see a series of lifecycle event hooks, and they are defined in the file. And that's really the structure. There's a version number, specify the OS, specify some file, source, and destination. But then the rest of the file is lifecycle hooks. All right, so let's take a look at an appspec file in the source editor, and it'll really tie all this together. All right, so this is an appspec file, and they can look very differently, depending on their contents, but they all have a version number. Now the catch is here, the version number is required, but the only allowable version is 0.0. Very strange, something we just have to accept. It's kind of reserved by CodeDeploy for future use. OS, pretty self-explanatory. It specifies the operating system. It could be Linux, could be Windows. It's required as well. Then we have files. Files specify the files that should be copied to the instance during the deployment, and we can have multiple source and destination pairs. But it's basically, hey, which files? Where are they? Where do they need to go? And the specifics of where, it's in our revision, where at in our revision. And then of course, the destination, where do we want to copy these files on the destination? And that must be a fully qualified path. Now there's also a permissions section, and notice we don't have one. So obviously, it's optional, but it's basically what files need permissions, what



permissions do we need to apply to those files as they're being copied to the destination? Then we have the hooks section. And if you've ever seen auto scaling hooks, pretty similar concept here. So we see we have specific lifecycle events, BeforeInstall, AfterInstall, ApplicationStart, ValidateService. And within these lifecycle events, we can have scripts run. So it's very handy, and of course, it's provisioning our application server and also managing the deployment of the application. Now, these are pretty self-explanatory other than maybe ValidateService. ValidateService is the last lifecycle event. And just think about that term, what are we doing? We are validating that our deployment completed successfully. Notice what we're doing. We have a MonitorService. Now I'll provide a link in the lesson materials to a lifecycle hooks for CodeDeploy page. I'd say maybe read through it one time and just get a general idea. It'll go into deeper detail on BeforeInstall, AfterInstall and all the hooks available. Certainly not something that needs to be memorized, but just have a general idea of what's going on with this. And this is certainly a representative example of what's going on and how an appspec is managing our deployments. Key point to remember that remember we have different types of deployments, deploying to EC2, deploying to Lambda, deploying to ECS. And they can all have different needs in the appspec file. So again, I'll provide the link, maybe review that page one time and just have a general idea of what's going on with each type of deployment. So that's a look at a pretty typical appspec file. All right, so we looked at an appspec file and what it can do for us. It essentially manages our deployment. If we have some custom actions that we want to do, we can certainly put those actions in a script in our appspec deployment file, and we can specify the lifecycle event that we want to run those custom actions. I specifically mention custom actions. That's sort of a key term. We always want to be focused on key terms and scenario questions as clues to the service we're talking about. So if we need to run custom actions during a deployment, we're thinking about our appspec file and more specifically lifecycle hooks in the appspec file. So that's a fairly in-depth look at the appspec file for CodeDeploy. And with that said, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodeDeploy Revision and Deployment Details

Hey, Cloud Gurus. Welcome. So in this lesson, CodeDeploy Revision and Deployment, sort of ties everything together. And we will talk about how we can deploy revisions and just really go through the step by step in how we will get our revision deployed out to our servers. And what is an application revision? Well, we've talked a bit about this. And what CodeDeploy needs to see here is a ZIP file or a TAR file, and I've fallen back on the Java example, a JAR file. And by this point you can probably guess where at least some of my background comes from. Yes, I've seen my share of JAR files, but CodeDeploy needs to grab that revision. Where is it grabbing that revision from? It's grabbing it from S3. And just



remember, we have other options for where we can store that revision. The most common is S3. So this is a two-step process. First of all, we just have to decide, hey, where do we want to store our revision? And let's say we choose S3. And then the second part, we need to bundle our revision, get it in that format, basically build our application, ZIP file, TAR file, JAR file. And thinking in terms of our Git commands, push that revision to our S3 bucket. And I'll give a sneak preview if you decide to go more in depth with the supplementary lesson. We're going to use `aws codedeploy push`, and that will bundle our source, push it to our S3 bucket. And just remember from the previous lesson that in addition to our bundle, our TAR file, ZIP file, whatever, we need that `appspec.yml` file as well. So, our bundled code will be in that same directory as our `appspec.yml` file. So again, it's just key to have this terminology down. And a revision is simply the bundle containing the current group of files that we want to deploy. And that word revision almost lends itself to thinking, hey, this is an update, and it certainly could be an update. It could be a bunch of bug fixes. We could be deploying for the first time. But all of those actions fall under that umbrella of being a revision. All right, now being at the end of the CodeDeploy section, this is a bit of a review. Let's fill in the blanks here. First of all, we need to select a repository type in which to store the bundled source code. And in this example, we select S3. And again, we bundle the source files and push them to the selected repository. And in the supplementary lesson, we will do just that. But just realize that in most cases, we're working from the command-line interface, and we are using CodeDeploy commands. So it's `aws`, `space`, `codedeploy`, then the command. In this case, it is the `push` command for CodeDeploy. Now obviously, that terminology is borrowed from GitHub. And yes, it is pretty much the same operation. We are pushing that bundled code, TAR, JAR, ZIP, whatever to our repository. So we've done that push command. We have our revision in our S3 bucket. How do we deploy this revision? Our code is in our S3 bucket. And that's where CodeDeploy can grab that revision and deploy that revision to our deployment group. Remember again, I just want to emphasize this point. It could be a point of confusion. That deployment group, it's a group, whether it's 1 EC2 instance or 100 EC2 instances. So don't be confused by the word group. One, in this case, is a group. And if you can get your mind around that, hey, one instance, hey, that's our deployment group. Just remember what about our deployment configuration? Remember our options, `deploy all at once`, `deploy half of the time`, `deploy one at a time`. Now, I'm speaking of doing a deployment to EC2. So the other little potential gotcha, well, if our deployment group is only one instance, the only valid option really would be `deploy all at once`. We have one instance, we deploy to it. Well, sort of by default, that's all at once. All right, let's talk about a few exam tips. And we've talked about the lifecycle event hooks for CodeDeploy. Similar concept to lifecycle events for Auto Scaling groups. And it makes a lot of sense, especially if we're deploying to EC2. And our hook section for an EC2 or on-premises deployment contains mappings that link to deployment lifecycle event hooks to one or more scripts. The hooks allow you to run custom code at various times during deployment, examples being



BeforeInstall, AfterInstall. And in the previous lesson, there's a link for just an overview on the AWS page for these lifecycle hooks. Now, as an example, we talked about other examples, but let's say we wanted to run custom verification actions after our new revision is deployed. We could use the lifecycle hook, AfterAllowTraffic, and within that hook, we could include our verification actions. So again, it's not mandatory to memorize every single one of those hooks, but certainly read that page, have a good understanding of what they're doing and what we're trying to accomplish with these lifecycle hooks. All right, some questions to consider, and these are sort of rhetorical questions. But basically, the overriding question is, are we ready to deploy? And if we can check off these answers sort of in a checklist, then we're ready to go. Have we created an application? Now not included on this page. Well, have we bundled a revision? That's sort of obvious. We certainly need our JAR, TAR, ZIP file, whatever to deploy. Do we have that? Have we created a deployment group? So, what are we deploying to? Are we deploying to one EC2 instance? Are we deploying to an Auto Scaling group of instances? Are we deploying to Lambda or ECS? So certainly, we have to have a deployment group. Have you created a deployment configuration? So, what's the process of our deployment? How quickly are we doing it? What is our level of risk? And by that, I mean, deploy one at a time is less risky than, hey, let's deploy all at once. We have to think about what if the deployment goes bad, and we have to think about rolling back. And just a quick quiz, is a deployment configuration mandatory? Think about that for a second. The answer to that is no. The deployment configuration is optional, but it's pretty darn valuable if you want to control the rate of your deployment. All right, now I'm going to back up just one slide here, and I want to emphasize these lifecycle hooks. There's a good chance you'll see something about these lifecycle hooks for CodeDeploy when you sit for the exam. So certainly give this page a look before you sit for the exam, and just understand what it's doing. It's not necessary to memorize anything. But our AppSpec file has a hooks section. And just realize that sure, there are similarities, but between ECS, EC2, and Lambda, they're going to have some different hooks. There's ECS. These flow diagrams are helpful. Notice Lambda has less hooks available than ECS. EC2 has a good many. Again, no need to memorize these, but just read through this and at least have your eyes on them once, and you'll recognize them when you see them. Notice in the example I used, AfterAllowTraffic, you can use this deployment lifecycle event to run tasks after our instance is registered with a load balancer. So we can get really specific to when we want to run our tasks during the lifecycle of our deployment. So again, check that page out. Just have a good idea of what's going on and sort of recognize some of these commands. No need to memorize all of them. All right, so in this lesson, we talked about CodeDeploy revisions and deploying our revisions and the steps we need to perform in deploying those revisions. Remember these questions and have a good understanding of what lifecycle hooks are doing in our appspec.yml file. And with that, I'm going to end this lesson. Thanks for joining me, Cloud Gurus. I will see you in the next lesson.



# AWS CodeDeploy with Lambda

Hey, Cloud Gurus. Welcome to this lesson, Code Deploy with Lambda. So we've seen a lot of how to deploy to EC2 instances using CodeDeploy. Let's see how this works with Lambda. And let's talk about our deployment workflow on Lambda. We create an application, we specify a deployment group, specify a deployment configuration. We need an appspec file, then we can deploy. Check out our results. And if needed, we can redeploy. And maybe we need to make some changes to our appspec file to redeploy. And here's a look at a typical appspec file for a lambda function. In fact, we're going to use this appspec file in the demo with the minor alteration of removing the Hooks section. We'll keep it simple. We're going to have two versions of a lambda function, and this appspec file, again minus the hooks, will do the trick for us. Now not to totally ignore hooks, let's talk about those a little bit in the appspec file. What are we doing with a lambda function in hooks? So in our previous appspec file, we had a Hooks section, BeforeAllowTraffic: 'LambdaFunctionToValidateBeforeTrafficShift' and AfterAllowTraffic. So, these hooks are very literal, and we get a good idea just from the name of the hooks. But note above highlighted in red, either before traffic or after traffic is allowed, the lambda functions are performing validation. Validate before traffic, then see what it looks like after traffic, and we can validate again. And what's this process look like? Well, of course, we start, and we have our hook, BeforeAllowTraffic, and our validation will execute. And say things go well. We then allow traffic. We have our other hook, AfterAllowTraffic, do some more validation. And that's the end of the cycle. Now, we have items in gray, and those items in gray cannot be scripted. They're not hooks. We're working with BeforeAllowTraffic and AfterAllowTraffic. So let's see how this all looks with a little demo, CodeDeploy with Lambda. All right, so this is sort of a conversation. I like what CodeBuild can do for me, but I can quickly use CodeDeploy to deploy a new version of my lambda function? Sure, absolutely. And let's do a demo on that. So we're going to have a version 1 of a lambda function and a version 2. We will also have an alias. We'll call it Staging. Initially, it will be pointing to version 1. But in our demo, ultimately using CodeDeploy, we'll see how we can deploy version 2 and shift the alias to that new version. All right, out at the Management Console, just to save a little bit of time, what do we need for a lambda function? We need an execution role. So I went ahead and created that named cd, cd for CodeDeploy, LambdaRole. Notice the policy, AWSCodeDeployRoleForLambda. That's created already. Let's go back to Roles, just one step. Create role, AWS service, CodeDeploy, just want to make sure we know how to do this, and then CodeDeploy for Lambda. And that's how we'll get that policy that we need, but that's done. So I'll cancel out of that. All right, and one last thing. Let's go back to Roles. Let's find our role, and let's copy and paste our ARN. Save that somewhere. Just make sure you don't copy over that. We're going to need that ARN. Now let's go to Lambda. I'm in Functions. I want to create a function. Author from scratch. Just call it cdLambdaFunction. I'm going to go with Python, and



Create function. All right, so we have our code, and I'm just going to change this little message. I'll just call it Demo for Lambda! Just a change for change sake. And when we change this code or deploy, change is not deployed. So let's click Deploy. And this little bit of code here, we're going to call that version 1. So let's click the Versions tab, and click Publish new version. Call it version 1, and click Publish. Let's go back to our function, back into our Code tab. And we decide to make our description more accurate. And again, this is a simple demo. But of course, this could be much more complicated code. But nonetheless, this is a code change. Let's deploy, go to Versions, Publish new version, call that version 2, and Publish. All right, next let's create an alias. Let's go back to the function, go to the Aliases tab, click Create alias. Now remember in our diagram, we called it Staging. And for now, we'll point that to version 1 of our lambda function. So let's click Save. So we have two versions and an alias. Now let's go to CodeDeploy, and we want to create an application. Just give the application a name. Compute platform, AWS Lambda. Just noting the other choices, but of course, it's Lambda for this demo. Create application. Next, we want to create a deployment group. Remember our service role and that ARN. It's either in your clipboard or just paste it in a document. We want to plug that in. Could paste it in, but it's right here as well. And we'll stick to Lambda all at once. Click Create deployment group. All right, next, we want to create a deployment. Revision type. Well, we don't have an application stored in S3. Let's Use AppSpec editor. And remember from the slide, our AppSpec, our appspec file minus those hooks, CurrentVersion: "1", TargetVersion: "2". Now, it looks a little off here. We need the name of our lambda function. These are just fillers and the alias. Just recall, that's the name of our function, and our alias is Staging. All right, so that's our appspec file, specifying the lambda function, the Alias, Name, CurrentVersion, and where do we want to deploy our TargetVersion? All right, now I did one thing wrong. Can you see it? Well, this is YAML. Just need to make sure YAML is clicked. All right, now we're good. I can click Create deployment, deployment created. Ah, that didn't take long. Traffic shifting progress. All traffic is shifted to the updated lambda functions at once. Remember that all-at-once deployment. But long story short, here's version 1, here's version 2. We have deployed Lambda version 2. Let's verify. Let's go over to Lambda. Remember our alias. What's going on here? It's saying version 1. Well, we just need to refresh. Ah, version 2. So we have successfully used CodeDeploy and Lambda versions and aliases to deploy a new version of our lambda function. Maybe most importantly, point our alias to that version, to that new version. And that's a look at deploying lambda functions with CodeDeploy. And with that, I'll end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodePipeline



# Introduction to CodePipeline Concepts

Hey, Cloud Gurus. Welcome. So in this lesson, Introduction to CodePipeline Concepts, we're going to tie everything together from this section. We see the makings of a pipeline. How do we manage all of this? But before going much further, let's answer the question, what is CodePipeline? Well, it's a continuous delivery service that we can use to model, visualize, and automate the steps required to release our software. And with CodePipeline, we can quickly model and configure the different stages of a software release process. Let's first talk about some CodePipeline concepts. There's that term, continuous delivery, automation, build, test, deploy, setup and config. All right, so what about continuous delivery? Well, CodePipeline is a continuous delivery service. Just keep in mind, we can add in manual approvals, and that plays into automation. Again, how far do we want to automate? And it's back to that continuous deployment versus continuous delivery, but just realize that it's really what we want to do with CodePipeline. Yes, it helps us manage our pipeline. Ultimately, if CodePipeline had a job description, the first line in that job description would be automates our pipelines. Build, test, deploy, CodePipeline can help you automatically build, test, and deploy your applications. And it's sort of the framework for our pipeline. We deploy via this pipeline. And if we use CodeCommit, CodeBuild, CodeDeploy, the pipeline is the vehicle by which we're automating our code and getting it out into our environments, build, test deploy our prod environments, ultimately. Now let's talk about setup and configuration, and we had sort of a preview of this, but it really becomes a prerequisite, not just for CodePipeline, but for all of these services. Well, we need an IAM user, and I don't have a lot of detail here, but obviously that user is going to need permissions, permissions to CodePipeline. What else? Well, everything that's in the pipeline, CodeBuild, CodeDeploy, and that's where the policy comes into play. We apply that to this user. Now CodePipeline in the Management Console is very handy, and CodePipeline sort of lends itself to working in the Management Console. But that's not always going to be the case, and we can certainly create and manage our pipelines from the command-line interface. So, another part of setup and config, we install the AWS CLI. So, let's ask some more questions. Can we use SNS? Yes, we can. SNS can be tied to our approval actions. All right, we're talking about triggers. Can we use Lambda with CodePipeline? Absolutely, we can. Again, just understand these repeating patterns. EventBridge can be used with CodePipeline. EventBridge can trigger a lambda function. All right, let's take a look at a pipeline. And we've seen all of these stages, we just haven't seen them all tied together really in a pipeline. We have our source stage. Remember, CodeCommit, GitHub, Bitbucket. Our developers commit changes. And if we have a build stage, that will trigger a build. Our changes are built. Remember, the build stage is optional. It kind of depends on what kind of code we're working with. Java, yes. We need to build our Java code into a JAR file. We need a build stage. Straight HTML, we don't have



anything to build really. Staging, so understand that that encompasses deployment, CodeDeploy, maybe Jenkins, and testing. And we could integrate some testing services outside of AWS. Production, deployed to our public servers. Again, staging, production or testing and deployment, that's under the auspices of our deployment stage. I'll just say generically CodeDeploy. Now, it's good to understand the flow of our artifacts through this process. Source output artifacts, going to an S3 bucket. Our output from one stage becomes the input to another stage, to the next stage actually. Output artifacts from our source become input artifacts to our build. We perform our build. Remember where that goes. That goes into an S3 bucket. I'm showing one S3 bucket, and that's certainly allowable to have one S3 bucket. But we could, if we want to, segregate this, create multiple S3 buckets. All right, so we've built our code. We want to deploy. The output from our build becomes the input to deploying. And then finally, not showing on this diagram, we deploy our code to the environment we want to deploy to, maybe a testing server, maybe a beta server or the whole way out to prod. So with all that said, let's go out to the Management Console and walk through building a pipeline. We're not going to complete the whole process, but it will be helpful to see this in action. All right, so we're out at the Management Console on AWS, and we want to go to CodePipeline. Now we're not going to create a pipeline. We'll do that in upcoming lessons, but let's just walk through the creation of a pipeline and sort of build on this knowledge as we go throughout this section. So let's click Create pipeline. Give our pipeline a name. All right, Pipeline type. Which pipeline is right for me? There's some information. Basically, version 2 allows you to add additional parameters such as triggers on Git tags and pipeline-level variables. More on that later. We just want to keep this demo a little bit more basic. All right, so we're given a new service role, and we'll just go with that. We can add variables at the pipeline level, and there's a page on these variables, which I'll include in the lesson materials. But let's scroll down to Use cases and just take a read through these use cases for variables. It pretty much explains what they're all about. Although again, we'll look at those in an upcoming lesson. Advanced settings, all right, where are we storing our artifacts, default or custom location? Basically, S3 bucket where we want to store our build artifacts. And what kind of encryption are we doing? Let's click Next. I'm going to close this pipeline settings. And here are the infamous by now drop-downs I talk about. Really study these drop-downs. It just really gives you an idea of what you can do in each stage of a pipeline. So all of these source providers are valid. And of course, with CodeCommit, that is an AWS-managed service. That's one of the advantages. Repository name, I'll just select the repository we've created and have been working with. And I'll just select main. How are we dealing with events and detecting these events? CloudWatch Events is recommended. Note the difference. CloudWatch Events automatically start my pipeline when a change occurs, whereas CodePipeline checks periodically for changes. Output artifact format, not terribly important at this point. Let's click Next. Note our stages then become optional. So we, of course, need to have a repository. Do we need a build



stage? Do we need a deploy stage? They're both optional with the caveat that you have to have one, otherwise you don't really have a pipeline. So let's open our build drop-down. Our options are CodeBuild and Jenkins, and then we can create or select a build project. We can add environment variables. We can do a single build or batch builds. I just wanted to take a look at this. Let's say we skip the build stage, which is perfectly allowable. And then we have a deploy stage. Again, look at all the options we have and study these drop-downs for deploy providers. And I'll just say CodeDeploy. Choose an application name, which, of course, you have to create in CodeDeploy. And then our Deployment group, to where are we deploying our code? And remember, that can be EC2, Lambda or ECS. With that done, okay, so we've essentially built a pipeline. And then we can just check our settings and create our pipeline. And I'm going to hold off on that. We'll go further and build on this in upcoming lessons. But those are the steps in building a pipeline. And just recall the flexibility involved in building these pipelines, several different source providers, build providers, deployment providers and the option of the stages you want to include. So that's a look at creating new pipelines in CodePipeline. All right, so we've looked at CodePipeline a bit, the key concepts, and went to the Management Console and just saw how we would build a pipeline. We saw all of our options. Study those drop-downs, and let's just look at some exam tips. What service is involved in position 1? It's the source. And when I say service, think CodeCommit. But also think, hey, it can be GitHub, it can be Bitbucket. What about position 2 build stage, CodeBuild or Jenkins? Just always remember, Jenkins needs an AWS plugin. It seems to come up a lot. What about position 3, staging? Testing and deployment, so we're thinking CodeDeploy. Also, remember again, that could be Jenkins as well. And four, we're deploying out to production, CodeDeploy. And of course, just realize that CodePipeline is managing this whole pipeline and more importantly, automating from start to finish. And there's our bonus tip to take away from this, so be prepared for that. Hey, GitHub, you might see GitHub in a scenario question. That is legitimate. You might see Jenkins in a scenario question. That's legitimate as well. And GitHub, why would we use CodeCommit over GitHub? Well, CodeCommit's fully managed, so we don't have to worry about servers, scaling those servers, and managing those servers. So that is an introductory look at CodePipeline, and we're going to go deeper in the upcoming lessons. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## CodePipeline with CodeBuild

Hey, Cloud Gurus. Welcome. In this lesson, CodePipeline with CodeBuild, we're going to go in and set up a pipeline in the Management Console, and we're simply going to select a repository, and we're going to use CodeBuild and perform a build. And this really emphasizes the point that we have options in creating our pipeline. The build and the deploy stages are optional. Of course, we



need at least one of those or we can have both. All right, so to emphasize that point, this is what our pipeline is going to look like. We're going to have a source and a build stage. Now for the source, we're going to use an S3 bucket. And of course, we could use CodeCommit, we could use Bitbucket. Just remember, for S3, to use as a repository, we need to turn on versioning. And I've already set up a bucket, and I did turn on versioning, so we're good there. And I intentionally left the diagram a bit sparse, but, again, it emphasizes the point. This is a pipeline. It's a two-stage pipeline. All right, we talked about this already. Have we turned on versioning? Well, for S3 buckets, that's what we need to do for our repository to be a repository. And just to get in to a little bit of detail about that, when S3 is the source, it's using an object key. So, by turning on versioning, we're ensuring that that object key is going to have a difference, and the difference is the version number. Why do we need versioning? There's a sort of long drawn-out answer, but basically the pipeline needs to have a way to identify which version of the artifact is tied to which execution. A pipeline can have multiple executions at the same time. Do we want to automate deployments right now? So when we create our pipeline, do we want to automate the whole way out? And we have that choice. So what this is getting at is that hey, we can set up our pipeline. We don't want to automate the whole way out to prod right now. Let's test this thing out a little bit. And then when we get it stable, we can go ahead and automate the whole way out to prod. Now in this case, our pipeline is just going to be source and build. Now in most cases, we would want to do a deploy, and eventually sure, we can add that. We can add the deployment stage, and we can deploy out to prod or deploy to wherever we want to go, maybe a beta server. But we can certainly build our pipeline in stages and add to it. Other things we can add, we can add manual approvals between certain stages. Again, the point is the flexibility. We don't have to do everything at once, and we can build as we go along and gain confidence in our pipeline. Of course, we know the answer to this. Can we choose the stages of our pipeline? Absolutely, but it must have at least two stages. If it doesn't have a build stage, it has to have a deploy stage and vice versa. All right, so let's bounce out to the Management Console and build a pipeline with a source stage and a build stage. So let's set up a pipeline with an S3 repository, and we're going to do a build. So we will have a build stage in our pipeline. Now, before we get started, what are we going to build? Well, we're going to use this MessageUtil.zip, and this needs to be in our S3 bucket, our repository. And expanded, this is what we're looking at, this pom.xml. That is for Maven. We're deploying Java projects. We certainly don't want to get into too much Java code in this course. But hey, we're DevOps engineers, and all we need to know is enough to be able to deploy this application. Let's take a quick look at the buildspec that is bundled in our ZIP file. So it has multiple phases, currently not using some of these phases. But let's say we're in a testing phase. And we anticipate some of these phases, install, pre\_build. We do have the build, and there's our mvn install. Again, we're building a Java project, and basically that's it. Just specifying where our artifacts go, but this is the key line of code in our buildspec. So with all of the preliminaries out of the way, go to CodePipeline in



the Management Console. But before we get started, let's take a quick peek at CodeBuild. And I have created a build project, basically specifying our repository, but we'll do that again in the pipeline. Click on the Build details tab. Here's the information on our build. I guess the key information is, all right, where are we deploying to? Well, an EC2 instance with Linux operating system. Note, using the buildspec, which we just looked at in the source code root directory that's bundled up with our ZIP file. So that is a key point as well. All right, so let's go ahead and do this. We click Create pipeline. Give our pipeline a name, and we'll just stick with the defaults here. Default service role. Keep it simple, no variables. Defaults in the Advanced settings. Let's click Next. Source provider, of course, is S3. And we have an S3 bucket, and there's our S3 bucket. Let's pop over to S3 real quick and just review that. There's our S3 bucket. Our S3 bucket is where we place our code. In this case, the MessageUtil.zip. A key point here is for a bucket to be a repository, we need to enable bucket versioning, and that's done. Our object key, what are we building? And note the little note here. In our case, it's MessageUtil. That's it. And we'll stick with CloudWatch Events. Click Next. Build, stage, absolutely. CodeBuild. What is our project name? It is this one that we just looked at. Let's click Next. We're going to skip the deploy stage. Go ahead and review, and let's create our pipeline. And with our nice Control Panel, we can just review the progress. It's grabbing our code. That succeeded. Now it transitions to the build stage. Of course, if we're in a testing mode, maybe we would disable this transition. But we have faith, we go right into the build. All right, so we have a Succeeded message. That took about a minute. Let's view the details. And if our build failed, this would be the first place to look. Let's go to Logs, and we have a real detailed log of our build. If the build failed, you'd be looking for an error message or multiple error messages in red. We had success. I don't anticipate seeing any red. Note the phases corresponding to our buildspec file. Notice it grabbing some Maven plugins. We don't need to know too much about that. A lot of information. These log files are pretty big, but we had success, and we have successfully use CodePipeline to build a Java project. And again, if you're troubleshooting, this is the first place to look. Look for some red error messages. So again, we did successfully set up a pipeline with a source stage and a build stage using S3 as our source stage, CodeBuild as our build stage. Three things to remember about CodeBuild. Well, first of all garbage in, garbage out. There's a problem within our ZIP file, something's out of version. We'll get that sorted out. But other things to remember, CodeBuild can be used in both build and test stages of a pipeline as can Jenkins. And with that said, we can add a CodeBuild action to a pipeline, as well as a CodeBuild test action. A build action and a test action, they can both be executed by CodeBuild. In this way, our entire build test process can be automated in CodePipeline, the build/test process contained within CodeBuild or Jenkins. So with that said, I'm going to end this lesson, and this lesson sort of open ended. We have a little bit of a troubleshooting exercise that we will address. I'll give us some time to think about that, but we'll get that resolved as we continue learning about CodePipeline and eventually some troubleshooting and some rolling back. So with that, I'm



going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## Managing CodePipeline

Hey, Cloud Gurus. Welcome back. In this lesson, Managing CodePipeline, we will see how we can do just that from the Management Console. Now we can also manage pipelines from the CLI. And there's actually going to be a supplementary lesson in this lesson materials, Managing Pipelines From the CLI. So when we think of our pipelines, we have a lot going on here. We could have source, build, and deploy stages or one of build or deploy. Don't forget our inputs to our pipeline and our outputs, and remember all of our input and output artifacts. And some typical things with our source, first of all, are we just referencing our source in the proper location? If we don't, we'll get an error when our pipeline is building. Other issues might be, hey, have we versioned our S3 bucket? And then what about our outputs? Go back to the basics. Our pipeline has to have the proper service role set up. And of course, when we go through setting up our pipeline, we do that. So when we start building pipelines, it's a good idea for us to know our way around the interface, know what we're doing, know what we want to do and know how to diagnose issues. So with that said, let's switch out to the Management Console. I'm also going to bring up a code editor and just show the fix for our issue in the last lesson. All right, so in the last lesson, we built our pipeline. And then when the pipeline is built successfully, it automatically executes. So it went and grabbed our code from the S3 bucket. That was successful. That was the first stage. Then it transitioned to the next stage, the build stage, and it attempted to perform a build. And as we saw, we got that error message. And the fix was very simple. So basically, we were using Maven. Looking at our buildspec, mvn, that's Maven. That's an Apache tool. So we're doing a Maven install. And basically, we had version 1.5 of Maven. So the fix to that issue was simply updating the version of Maven. And it's done with this bit of code here. That fixed the issue. So I'll switch out to the Management Console. All right, I'm at the Management Console, and I'm in CodePipeline. In the last lesson, we had created this devopsprodemo pipeline. So click on that. Now our pipeline ran successfully, but what if we had a problem? What if we had a problem in the source stage? Well, we can unit test. Essentially, we could disable this transition. And Unit Testing as a reason, and Disable. So, just this stage will run. And then we can work on it, check our error messages, and try to figure this out before completing our pipeline. How do we do that? Well, we just click Release change, and we can click Release. And it's just going to run this stage of our pipeline. Now we don't anticipate any problems. But if we had problems, we'd view the details, check out the logs, see what kind of error messages we're getting, and troubleshoot and fix it. And let's say we did that. We troubleshooted and fixed our pipeline. Everything is good to go. We could reenableView our transition. So let's go ahead and reenableView that transition. And notice, it



automatically started on our build. It's already grabbed the source, and it's starting on the build. Now one thing we could do is stop execution. Hey, something's not looking right here. Let's just stop. We select the execution. Well, that's the one we have in progress. Stop and wait, or we could stop and abandon. Just wanted to show this. I'm not going to stop it actually. And we back out of that. Our build succeeded, as expected. Now in the last lesson, we viewed these details and log files, but that's really the process you want to take. Even if your build succeeded, check this out. There's a lot of good information in here. Recall our stages from our buildspec file. A lot of Maven information in here. And I would say when you first start doing this a few times, just peruse this information and get an idea of what's going on in this build stage. Now in addition to viewing those log details, we could bounce over to CodeBuild. And lo and behold, our build logs are here as well. Same information. Phase details. Again, this is something good to familiarize yourself with. Note the different stages, and we didn't get any error messages. Everything succeeded. Some other information, no reports, no environment variables, build details. And this is really the build details of our build project. Linux EC2. Using the buildspec in the source code root directory, that buildspec is packaged up in our ZIP file. And there's our EC2 that we deployed to. And we could view that in CloudWatch. All right, let's go back to Build details. I just want to illustrate how there are alternate paths to get sort of to the same place. Artifacts. Well, here's the URL for artifacts of our pipeline. Where do our artifacts go? Well, S3. And we didn't specify any artifacts, but this is where our artifacts would go. Let's go back over to CodePipeline. And let's go back into our pipeline. And we looked at stop execution. What about notify? Well, you might guess where this is going. Let's just click Create notification rule. I'm not going to do this, but we'll walk through it. Look at all these events that can trigger an execution and all the different options. So we get down to the stage level or the action level or the overall pipeline level. We'll talk about manual approval, upcoming. We can create targets. Well, probably guessed this one, SNS topic, or we could go to Slack or Microsoft Teams. So we certainly have automations to notify our team of the pipeline status. Now just discard this. So we've had good success with this pipeline. Maybe we want to reuse it. Hey, let's go ahead and clone this pipeline. We'll create a new service role. This looks familiar, just presented in it a little different way. The default location, the encryption key. Let's clone our pipeline. That didn't take long at all. The naming is pretty predictable. Well, guess what? Our pipeline is attached to S3. It's never gone through this, so this triggers the action. It grabbed the source, and, of course, it's proceeding on to the build process. So remember, upon that first creation of the pipeline, it's going to execute. And then subsequent changes, we add a new build or we add new code, is going to trigger our pipeline. And that didn't take long at all. We have success again. This is the same exact pipeline, just a different name. Why do we want that? Well, it's sort of a starter pipeline for us. Let's click Edit. We can add to this. Essentially, that's the idea. We could change the pipeline type V1, V2. We could add variables. We could add to our stages or add additional stages. And let's just



do that. I'll just call it demodeploy. Click Add action, deploy pipeline. So obviously, we're going in a deploy direction, but look at all the actions we could add. We could add Manual approval, we could add build, and, of course, deploy. We could invoke Lambda or Step Functions. We could add a test stage. Here are our testing options. Of course, our source, we've seen that. Invoke deploy. I'll just say, hey, let's add a deploy action. Input artifacts, remember the output from our build can be the input to our deploy stage. We can add an application. I'm just not going to go the whole way through this. Deployment group, what kind of hardware are we deploying to or lambda function? And we would have successfully added that stage. I'm just going to cancel out of that. We still have the stage, we just don't have it doing anything. So I wanted to show that extra stage, but I'm not going to do a deploy stage, sort of a spoiler for upcoming lessons. Well, let's go ahead and delete that stage. A disclaimer, yeah, we want to delete, and we're back to where we started with our clone pipeline. And let's wrap up by going back to our main pipeline and devopsprodemo-clone. Well, it was just a demo. Last, but not least, we can delete a pipeline. I'm going to keep this hanging around for now. Maybe we'll use it again. But if we wanted to, we could delete that pipeline. So that's a look at managing a pipeline from the AWS Management Console. All right, what can we do with a pipeline after creation? Well, certainly view it, view the log files, most importantly, make sure everything was clean, and troubleshoot. Look for the red text. That will be your error messages if there is a problem. We can edit our pipeline. We can add stages. We can add testing stages or deploy stages, all kinds of stages. Disable/enable transitions. And again, think of a use case. Hey, we have a problem in this stage. Let's isolate it, test it, and diagnose and fix the problem. And we can certainly retry. Remember release changes. Now, that's not limited to, hey, we actually have an updated release. It can also be a case where hey, we want to try this out again, and it's going to grab the latest release. And of course, we can delete a stage. Just remember, our pipeline transition scenarios. Hey, what happened here? Why is my pipeline not completing? Well, there certainly could be an error, but just remember, did you disable a transition? So if you disable, get your problem fixed, always remember to enable your transitions. And this is easy. I want to test the front end of a pipeline, but don't want to deploy. We've already done this, disable a transition to the deployment stage until you complete testing. So that's a look at how we can manage a pipeline from the Management Console. Again, check out that supplementary lesson on managing from the CLI if it interests you. But with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## Creating a Pipeline - Demo

Hey, Cloud Gurus. Welcome. So in this lesson, Creating a Pipeline, we're going to do just that. The difference is we're going to create a pipeline with a deployment stage and no build stage. So what's that sort of signal to us? Well, we're deploying



code, which we don't really need to build. Basically, we're going to deploy some HTML files, so we can go ahead and skip the build stage. So what stages do we want to include? Well, we've kind of already answered that. Of course, we're going to have a source stage, and we have our options. For repositories, we're going to use S3. Now I'm showing a build stage, dash lines. It's optional, of course. And in this case, we just don't need it. And our deployment group, remember CodeDeploy is going to be EC2. And just as a refresher, remember our source values. Remember those drop-downs. Build can be CodeBuild and Jenkins and many options. And in our case, we're going to use CodeDeploy. All right, so this isn't overly complicated, but there are several steps. So, let's just go out and get at it. All right, we've got a lot to do in this demo. To save a little bit of time, I've created an S3 bucket, and it's going to hold our code. And I've uploaded SampleApp\_Linux.zip. And just remember that our S3 bucket is going to be a repository, so we want to enable versioning. And I've done that already. All right, now think about what we're doing. We're going to use CodeDeploy. We're going to deploy our application. Where are we deploying it? Well, we're deploying it to EC2. So we have a bit of work to do in EC2. But first, let's go to IAM. And we want to create a role. Give our role a name, and Create role. We need to select the use case. Well, it's EC2. Allow EC2 to call services on your behalf. Let's click Next. And of course, we're working with CodeDeploy. So we want to attach a CodeDeploy policy. AmazonEC2RoleforAWSCodeDeploy is the policy that we want. Let's click Next. Give it a name, and just jot that name down. Make sure you remember it, create the role. Now we can go over to EC2 and create our instance. We want to launch an instance, give a name to the instance, and we want Amazon Linux. We're going to proceed without a key pair. Don't have any intentions of SSHing into this instance. Let's edit Network settings. We want to auto-assign public IP. Well, that's enabled, so we're good there. Now we want to go into Advanced details. We have a few things to do there. There's an instance profile in this drop-down that has the same name as the role we just created. And we want to attach that role to our instance. Now we have some software to install on the instance, and that's going to take place in User data. In User data, these are the commands that we want to place in User data. I'll include this script with the lesson materials, but basically we're installing some utilities we need, some software into the EC2 home, and we're getting the latest version of the CodeDeploy agent. This is an S3 bucket in us-east-1, and I'll include the link for the details to this as well, setting some permissions, and then performing the install. Okay, let's launch our instance, and this is our instance launching. I have two of these with the same name, but note that one has caps. So we're all lowercase, codedeployinstance. And our instance is running. Now we need to go over to CodeDeploy, get to this main page, and let's create an application. We'll just call it pipelinedeploydemo. Choose a platform. Remember our options. And of course, we're using EC2. All right, let's create our application. And going through our mental checklist or even better, have a written-down checklist, we need a service role for CodeDeploy. So let's go back over to IAM. And in IAM, let's go to Roles, Create role. And we need to choose a service or use case, and it's, of



course, CodeDeploy. Let's click Next, AWSCodeDeployRole. It's just an idea of everything that it's doing if you want to check that out. Let's click Next. Give it a name. I have a few of these. I'm just going to attach a date identifier. Let's create our role. Now we can go back to CodeDeploy, and we want to create a deployment group. Give it a name. Service role, well we just created that one. We'll do an in-place deployment to Amazon EC2. Now, recall the tag that we added to our instance, Name, and it was lowercase codedeployinstance. Now down here at the bottom, Load balancer. By default, we're enabling load balancer, and that's generally a good thing, but just doing a demo. Uncheck that. Let's create our deployment group. So we have an application, we have a deployment group. We're good to go with CodeDeploy. Now we create the pipeline. All right, we're just going to go with all these defaults, and we want to click Next. Get to the Add source stage. S3. I'm going to bounce over to S3 for a quick sanity check. This is our S3 bucket, pipelinedemo117. There it is. Object key, that is our ZIP file, and that's the name of it, SampleApp\_Linux.zip. Let's click Next. Last lesson, we did a build stage and skipped the deploy stage. This time, we'll skip the build stage, and we want a deploy stage. Our provider is, of course, CodeDeploy. Application name. And it doesn't hurt to double-check over at CodeDeploy. Here it is, pipelinedeploydemo, this one. Deployment group, we just created that. All right, we're ready to go. Click Next, and we can review our settings and finally create our pipeline. So of course, it's grabbing our code first. That succeeded. Now it's doing the deploy. All right, our deploy succeeded. That took less than a minute. And of course, we could view the details and go into our logs, but we've done that already. We have successfully deployed our application. Just to recap, we used S3 bucket, and remember to turn on versioning if you're using that as your repository. Skip the build stage this time. Went straight to deploy, which means that we have to create an application, have to create a deployment group, in our case, an EC2 instance. And remember the service roles necessary to delegate the proper authority to perform these actions. All right, let's talk about some quick exam tips. Creating a pipeline is the perfect time to get in and play around. Really review those drop-downs of the stages to understand the services available for each stage. See where Jenkins comes in to play and CodeDeploy and all that good stuff, see our different repositories. All right, one more tip. There we go with Jenkins. Understand Jenkins in a pipeline. Why use Jenkins? It's well suited for DevOps and agile methodologies that use quick iteration cycles. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## CodePipeline Continuous Delivery and Approval Actions

Hey, Cloud Gurus. Welcome. In this lesson, we're going to talk about CodePipeline Continuous Delivery and Approval Actions. So a big focus of the course, and this section specifically, is on automation. And it's great to be able to automate the



whole way out, the whole way out to our prod environment. But that may not always be the best option. There are times where we may want to just put a stop sign there before we go the whole way out to prod. Get some folks to review what we've done, approve it, and then move our code out to prod. So yes, we can fully automate. It's a great thing, but do we want to always fully automate? And if not, why? Well, maybe we've had a release that's buggy. So instead of pushing the whole way out, let's test it. Sure, we can fix those bugs and push out a release, but how can we be proactive about it? Well, we can set up approval actions in our pipeline, and we can set those up at different points in the pipeline. Are we ready to build? Add an approval action. Maybe we need to do a code review, then approve it, continue on. What about deploy? Are we ready? Let's stop, add an approval action, review, approve it manually, then do our push. So full automation is a great thing, but it's not the be all and end all, and we can add approval actions. Remember the transitions in our stages. We can add approval actions. Now one thing to consider, do we want just anybody approving and then pushing out our code? As you might expect, the answer is no, we can deal with that in IAM. We can attach a policy to a user or better yet, group of users, and that will give those folks permission to manage approval actions in our pipelines. We'll begin to see repetition in some of these patterns here. We want an approval in our pipeline. How do we notify the appropriate personnel? We can set up SNS notifications, and we can set them up for when they approve as well, SNS notifications. And remember our repeatable processes? What about lambda function notifying Slack? So keep those in mind, the repeating patterns, as we move throughout this course. Approval permissions, who can approve? We don't want just anyone or everyone to be able to approve, so we can set up approvers in IAM. A managed policy, AWS CodePipeline approver access. And always think about your IAM best practices. Do we have more than one approver? Let's set up a group, attach that policy to a group of users. Let's go out to the Management Console and set up an approval action. All right, so let's start with our most recent pipeline we've been working with, the pipeline with a source stage and a deploy stage. Okay, so how do we add stages? Well, let's go to Edit. And notice, we can add stages, full stages, or let's edit this deploy stage. We can add actions within our stages, and let's look at that. So let's add an action, and let's call it testdeploy. Action provider, let's do a Manual approval. We can add an SNS topic. Everything else is optional, and click Done. So, we perform a deploy, and let's say we're deploying to an intermediate level, maybe a test stage. Then we do a manual approval. We have folks review that deploy, then what? Well, we can add another action. So we want deploytoprod, using CodeDeploy. Add our application and our deployment group. Done. So, we grab our source. We perform a deploy, and let's say we're deploying to a test server. We need a manual approval here. Our QA team checks that out, approves it. Guess what? The automation starts back up, and we deploy to prod. So that's one way we can do it. And I'm just going to delete these and get back to where we were. Now, we can add a stage. Let's say we want to add a stage between source and deploy. I'll call it TestandQA. So when our automation kicks off, it's going to



grab the source and prepare it. But wait, we want to test here. Our QA team checks out the source code. They need to sign off on this. When they do that manually, the automation resumes, and the deploy kicks in and completes. I forgot something here. Think about what did I forget? It's probably pretty obvious. Well, our stages need an action. So of course, I need to add an action, and I'm going to call that manualapproval. Action provider, Manual approval. SNS topic, I'm just going to grab one of these. Remember, we have to subscribe to that topic and most likely get an email. It depends on what provider we select, and then we're done and ready. All right, so let's test this out. I'll click Save. And then how do we kick this off? Well, we can select Release change. Now remember, a push to our S3 repository would kick off our pipeline as well. It's grabbing the source. That succeeded. It moves on to TestandQA. It says Pending right now. What's it pending? Well, it's waiting that manual approval. So behind the scenes, our QA team is subscribed to that SNS topic. They get an email most likely. We could set it up with Lambda to send a Slack message as well. So our QA team gets that message. They come in and do a review, and let's say they reviewed everything. It looks great. They approve, submit. Approved, pipeline automation continues, and it goes on to deploy. Now it took a few seconds to update, but In progress. The deploy is happening, and our deploy succeeded. We added a manual approval to our pipeline, which stopped that automation and then resumed automation after the manual approval is completed. Now two key points before wrapping up the demo. So we have manual approval. QA team goes in. What if they reject the code? Well, it stops, our pipeline stops, and our dev team needs to go in and bug fix, and then we can restart our testing. Now, let's say that everything worked great and we're confident in our code. We want to get rid of this QA stage. Well, we just go back into Edit, edit TestandQA, Delete, Save, and we're back to having source and deploy automation. We know that works already. So we saw how to set up manual approval, both between stages and within a stage. Let's check out some exam tips. Let's remember our use cases, maybe a code review, again, between our source stage and whatever stage is next. It could be for QA and testing. That could be added anywhere before deploy stage. We could just be reviewing new or updated text before publishing. So just remember the use cases. The approval actions are pretty simple otherwise. And always remember our pattern of automation, CloudWatch Events, SNS, Slack, and Lambda. So with that, I will end this lesson. Thank you, Cloud Gurus. I will see you in the next lesson.

## CodePipeline with EventBridge

Hey, Cloud Gurus. Welcome back. So in this lesson, we're going to talk about CodePipeline with EventBridge. And just a caveat here, remember, EventBridge is a superset of CloudWatch Events. So we've already seen these events in action and specifically on the source stage. So we push code to our source repository, and our event can be watching that and can trigger our pipeline. Now



we realize that our events can do a lot. It can trigger SNS notifications. It can trigger a lambda function, and maybe we're updating a Slack channel on events. So just realize that our events can be at multiple places within our pipeline. Again, dev team commits changes, our event is watching that. It triggers our pipeline. It can trigger when changes are built or deployed or tested. If we have something going on on our server, we have some sort of event, we can create an SNS topic, subscribe to that topic, an EventBridge, and trigger that topic. And that pattern is reusable, and it can be reused in multiple stages. Likewise, with Lambda, we can certainly trigger Lambda and certainly do anything under the sun with Lambda. But a typical use case is to notify Slack. Another thing we could do with Lambda, okay, so we do a push to our source. Maybe there are supplementary actions we want to do. We can do that with a lambda function before we build. So just understand the versatility with our events and that they're not limited to a single stage. And in fact, we can work with events with actions within our stage. So to state that more officially, CodePipeline reports an event to CloudWatch Events whenever the state of a resource changes. And these events are admitted on a guaranteed at-least-once basis for pipeline executions, stage executions. And remember, we can have actions within those stages. Action execution is a great example. Let's say we have a deploy stage with multiple actions. So we deploy to test. We have an approval action, then we deploy to beta, have another approval action, final test on our code, have another action, final deployment. But our event can tag along in there anywhere and notify us of those events. So I just want to have a short lesson on this, our events. And just again emphasize, EventBridge is sort of a superset of CloudWatch Events. It has a bit more functionality. They're somewhat interchangeable. So let's go out to the Management Console and just check this out a little bit more. All right, out at the Management Console in CodePipeline, I'm not going to create a pipeline, but just step through to see some relevant points. Click Next. Provider, I'll select S3. And of course, we've seen this before, but really want to focus on it in this lesson. We use Amazon CloudWatch Events to automatically start pipeline when a change occurs. Now, in this instance, we can think of CloudWatch Events as EventBridge Events, and we'll see when we go out to EventBridge. An important point to note is that yes, when we push our code to S3 or CodeCommit, that is certainly an event that's going to trigger our pipeline. But there can be other events within our pipeline that we want to react to such as a build completing or maybe more importantly, a build failing. And we can trigger an event, and we can send a message to the appropriate team. All right, we don't need another pipeline, so I'll just cancel out of this. And we're going to focus on our DevOps Pro pipeline. But let's go over to EventBridge, and we want to be in Rules. I'm there already, and here's where we could navigate to it. All right, so within Rules, we have this codepipeline, devopspro-rule. Let's click on that, and let's click on Targets. And the target is that devopsprodemo pipeline. So when our event is triggered, it's going to trigger this pipeline. And we've seen that basically when we start a pipeline, but that's just the simulation really. When we do a push to an S3 bucket, that will trigger the



pipeline. Let's go back to that Rule, Event pattern. So, this isn't magic. Here's our event pattern. Our source, S3. We push our code to S3, and that's our trigger. And again, the target is our pipeline. Now recall the S3 bucket and the creation of the pipeline. So here's the bucket name. And the key, our ZIP file, MessageUtil.zip. So if we make changes to MessageUtil.zip, we make those changes locally, commit it, push it back to the S3 bucket, and that triggers our pipeline. All right, so let's take a step back from this. I'm going to click on Rules, and I'm going to click Create rule. Now recall, we didn't create this rule with our pipeline. All we did was leave that default radio button for what it says is CloudWatch Events. Remember, CloudWatch Events synonymous with EventBridge Events. So let me click Create rule. We can do this manually. Default bus. Rule with an event pattern. That's what we want. Click Next. AWS events or EventBridge partner events. We could have other custom events or just about any type of event. All right, so let's click the drop-down here and just see how powerful this is. There's just a huge list of services that could be involved in this. We want CodePipeline. Notice the options, Action Execution State Change. Recall in that last lesson, we have stages, and within those stages, we have actions. And then this is for pipeline execution, overall pipeline. I'll just go with Stage Execution. And we start to get some code, some JSON code. And if you're not real familiar with JSON, just hang in there. When we get into CloudFormation, you'll see a lot of JSON. And notice that it's calling this a sample event. You can reference the sample event. You can write an event pattern. You can test this out. Here's the important point about a sample event, you don't have to select or enter a sample event. It's recommended so you can reference it when writing and testing the event pattern or filter criteria. Now, we just need to scroll down here. Event pattern, again AWS services and this huge list that works with EventBridge. Let's select CodePipeline. Really, the same thing we were looking at above in the sample event. I'll go with State Execution Change. And keep an eye on this JSON. This is really an important part of this lesson. Watch when I change this. I'll say FAILED, a failed event. Notice how my event pattern code changes. Let me add SUCCEEDED, RESUMED, CANCELED. The point is, you may see this type of code on an exam, and just familiarize yourself with it. And you might have multiple choice, four options. And there may be a state, a key word that you want to pick out of the question, which one represents an event for failed action. In which case, you'd see something like that. And just the really key point of this is, hey, we already have an event watching our pipeline, but we can customize the events to meet our needs and again send messages maybe for a failed stage or a failed action. We want to notify somebody of that. We set up this event pattern, a FAILED state. Well, what do we do next? Think about that before I click Next. Targets. And notice a bunch of targets. Probably most relevant, SNS topic. So we send the event to an SNS topic. Maybe our dev team subscribes to that topic. We get a failed pipeline, a failed stage in our pipeline. Well, our dev ops team gets notified via email. We could do a lambda function and send it to a Slack channel for more immediate notification. Those are just some of the things we can do with this. Key point is, our pipeline can have more than one event. Final point to make



about this is notice, okay, so we have a lambda function as a target. Well, we're not restricted to one target. So we're doing something with a lambda function. Maybe we still want to do something with SNS. We can have multiple targets for these events. All right, exam tips. Well, that whole trip out to the Management Console was exam tips, but a couple important tips. When thinking of automation, think of services such as EventBridge, Lambda, SNS, and these patterns can be placed in various parts of our pipeline and reused really throughout AWS. Lambda needs to be triggered. EventBridge can provide that trigger and start many automations, both to Lambda and elsewhere, as we saw. Just realize what's going on here. We're in a transition period. EventBridge is a superset of CloudWatch. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## SDLC Concepts and Services

### Jenkins with AWS Developer Tools

Hey, Cloud Gurus. Welcome. In this lesson, we're going to talk about Jenkins with AWS Developer Tools. And Jenkins is a tool that you'll run into from time to time on the DevOps Pro exam. You don't have to go into real great detail, but you just have to have an overview knowledge of Jenkins, what it is, and how it can interact with our AWS Developer Tools. So we'll start the lesson just describing what Jenkins is, take an overview look at it, and we'll look at some deployments of Jenkins interacting with our AWS Developer Tools. And we'll go into some configuration on Jenkins, not too deep, but just enough that we understand how this works and how we configure it. Take a look at it interacting with CodePipeline, and we'll finish up with some exam tips. And when you start preparing for the DevOps Pro exam, you do get into time management and time spent on particular services. I decided not to do anything hands-on because maybe it doesn't warrant that much of an attention versus some of the AWS services. But again, let's know what it does and how it interacts with our AWS tools. All right, so what is Jenkins and how does it interact with our developer tools? Well, Jenkins is an open-source CI/CD tool. Now let's take a look at some of our developer tools. And I'm showing this overlaid on a pipeline, but you'll notice that CodePipeline is on here as well, which is somewhat redundant, but you'll see why I'm showing it in a second. So where does Jenkins come into play with all of this? First of all, and this is a key point, how does Jenkins interact with AWS services? And the answer is there are Jenkins plugins for AWS. So Jenkins plugin for CodeBuild, Jenkins plugin for CodePipeline, plugin for CodeDeploy, and several other plugins. Now we don't need to know all of the plugins, but the Jenkins on AWS whitepaper is a good whitepaper to scan through, see some of the deployments, see some of the plugins. Notice that this whitepaper is archived, but the information is still useful. So if we want to use Jenkins with our



AWS Developer Tools, we install one of these plugins. And what we're doing basically is Jenkins can replace CodeBuild, Jenkins can replace CodePipeline, and Jenkins can replace CodeDeploy. Now of course, we replace all of them. Well, then we're kind of doing a Jenkins deploy, but we can replace one or more of these with Jenkins. And again, this is an open-source CI/CD tool. So really, the key takeaways from this slide just to know for the exam, Jenkins is an open-source CI/CD tool. Jenkins can interact with our AWS Developer Tools using Jenkins plugins for AWS. All right, let's go a little bit deeper. And for AWS deployments, Jenkins is deployed in a master agent configuration or master worker. And in this particular deployment, the master and the worker nodes can be on the same EC2 instance. Now, as you would expect, we can scale this and separate the master from those worker nodes, depending on our compute needs. It is a scalable solution, workers on separate instances. And this is another key point to remember, just the scalable solution master worker scenarios. All right, so we can scale. What about high availability, durability? Definitely. We could have multiple masters in multiple AZs, and that can safeguard against any kind of AZ disasters. So those are some of the basics of Jenkins. Let's look at some configurations. All right, so we could start off with an Elastic IP address for our Jenkins server. And our workers, we could have in an Auto Scaling group. Now I'm not showing specifically Auto Scaling group symbol, but that's what it is. And what does our Auto Scaling group do for us with these build workers? Same concept as any Auto Scaling group. If we need more workers, we can scale out. If we don't need them, we can scale in. And again, I recommend doing a quick read through of that Jenkins whitepaper. It's pretty readable as far as whitepapers go. And this is one of the configurations, but there are several other in there. But certainly remember this setup, Jenkins are build workers in the Auto Scaling group. All Jenkins configurations need a jenkinsfile, which provides instructions for Jenkins. And this would sort of be analogous to think about some of our other developer tools. CodeBuild has a buildspec file. We also have AppSpec files, same sort of concept here. We're providing instructions for Jenkins. And let's take one look at a configuration. This is Jenkins with CodePipeline. So our developer commits code to AWS CodeCommit. And that's really a typical action that really kicks off a pipeline for us. AWS CodePipeline detects that new code and triggers a pipeline, nothing new here. We've seen that many times. Here's what gets a little new. AWS CodePipeline invokes Jenkins to build the application. So that's certainly different for us. And I'll say it again, Jenkins works with our developer tools via Jenkins on AWS plugins. All right, so we have a build using Jenkins. Well, we build our code for a reason. How about we deploy it? CodePipeline triggers deployment on AWS CodeDeploy. CodeDeploy deploys the application on to EC2, our deployment group here. And again, I'll say check out that Jenkins on AWS whitepaper. Pretty quick read, something you can skim and check out. There are several other configurations for Jenkins working with our developer tools. So Jenkins is a very robust and pretty common CI/CD tool. And that sort of answers the question, why use Jenkins with CodePipeline? Well, you may be in a Jenkins shop that has decided to start using AWS. And rather than



scrap everything, there are ways we can integrate the two. All right, so that's about as far as we need to go with Jenkins. Let's take a look at some exam tips. How could we integrate CodeBuild with existing Jenkins build jobs? Well, we could use the Jenkins plugin for CodeBuild to integrate, and this eliminates the need for us to provision, configure, and manage Jenkins build nodes. In other words, we have CodeBuild jobs already. This allows us to integrate. What stages of a pipeline does Jenkins operate in? Jenkins can operate in the build and/or deploy stages of a pipeline. It can also replace CodePipeline. So those are a few extra tips. Just remember, Jenkins is an open-source CI/CD tool. It can interact with our AWS Developer Tools and our deployment pipelines using Jenkins for AWS plugins. It can replace CodeBuild, CodeDeploy, and/or CodePipeline. So that's a look at Jenkins on AWS. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## Pipeline Deployment for Multi-Account Environments

Hey, Cloud Gurus. Welcome to this lesson, Pipeline Deployment for Multi-Account Environments. So in this lesson, we'll take an overview look at multi-account environments, and we'll check out a diagram where we implement this pattern. And finally, we'll go over some exam tips. So by now, we've had a lot of lessons on CodeCommit, CodeBuild, CodeDeploy, CodePipeline and hands-on labs as well. But multi-account environments and deploying to them is an important pattern that we should understand. So let's answer the question about what are multi-account environments and why we would use them. So we're just looking at a general pipeline here. And ultimately, what do we do with a pipeline? Well, we deploy our code. Now typically, we would think, well, we're deploying in the same account. So why would we do multi-account deployments? And just start thinking about this, thinking why this pattern would be helpful for us. And we'll sort of peel away the layers of the onion here. Well, we can apply appropriate security to each account. So let's say we have three accounts. What do you think those three accounts would be with respect to our deployment pipeline? Start thinking about that. We'll reveal here shortly. Why deploy to multiple accounts? Another reason, apply appropriate security for access to our environment. Ah, there's a hint. Maybe we're thinking about environments within our multiple accounts. Apply appropriate security on our database. Well, what database are we talking about here? All right, enough with the suspense. Probably guessed this, Account 1 is a dev account, Account 2 is a test account, Account 3 is a prod account. Now, what we can do with multi accounts is set up our security in such a way that hey, our dev account, notice the green box here, that doesn't have to be super restrictive. We want our developers to have some freedom here, freedom for creativity, lots of deployments, not too tight on our security. Then we have our test account, and this is an orange box. More restrictive. We don't want to go crazy on who has access to our test account, maybe a lead developer and some test leads. And as



you would expect, this is red, just pointing that out compared to the orange, this is red, very restrictive on our prod account. So why do we do multiple account deployments? We want to isolate accounts and limit the impact of errors to that account. In a sense, we're siloing our accounts, or you can think of these maybe just figuratively as firewalls. We can afford some errors on dev, we want to be a little bit tighter on test, and we want to be really restrictive on our prod account. So who should have access to the prod account? Well, think about that. Think about least privilege. So who are we giving access to this to? Well, realistically, there's going to be an admin who has access to the prod account. But as far as deployments, this is prod. We've got to be careful. We don't want a ton of errors. We don't want bad deployments. Let's give the pipeline access to our prod account. Now, here's where we need to just start thinking of patterns. How do we do this? And we'll answer that question on our next diagram. But remember, we're isolating this prod account, very limited access. This promotes least privilege to our prod database, and that's very key as well. Let's check out this pattern in action. All right, so we have two accounts, and we're setting up a deployment pipeline. There's our source, showing a Git repository. And just remember, think in patterns. This could be a Git repository, this could be CodeCommit. So our developers push code, and that activates our pipeline. Using CodeBuild can do an application build and also push our build package to an artifact store. I'm showing S3. Again, thinking patterns. This could be CodeArtifact. And then we're doing a deploy. Now, Account A is a dev environment. Account B is a prod environment, and just limit it to two accounts just to show a different look. This could just as easily be A, B, and C, dev, test, and prod. We don't want to get locked in to one school of thought. We're thinking about patterns. If we think in patterns, it's very helpful when we take the exam, because we're not given a diagram, we're given specifications. And if we can just think of, hey, I know this pattern, I know what's going on here, we're in pretty good shape. And another thing is, all right, let's get back to basics here. So our pipeline is deploying to another account. Well, how does it do that? Well, that's certainly back to basics. In AWS by this time, we know the answer to that. Our pipeline is going to assume a cross-account role. So it's going to assume this role and have access to deploy on prod. And the assumption of that role can be limited to our pipeline, again, least privilege. So we have our deployment package. That is retrieved during our deployment. Fetch those artifacts. We can deploy to dev. Our pipeline has assumed that cross-account role. And when it's time to deploy to prod, it can deploy in our second account cross-account access with that role. It's really a key to this pattern. So these multi-account deployments, it's an important pattern to know and understand, but we really can keep it simple and just think in terms of patterns. And getting cross-account access, how do we do that with our cross-account role? So, to wrap up, let's take a look at a couple exam tips. Why do we deploy to multi accounts? Isolation of errors and limiting impact. So, on the dev side, we have more freedom. It allows creativity for developers. But as we cross over to the prod side and maybe a test account in between, we get more restrictive. That's going to limit hands-on our



prod, deployment, and it's going to limit errors. So what is the key element for this pattern? Well, we go back to basics, multi-account environment, multi-account access. We just need a cross-account role. So that's a look at pipeline deployments in multi-account environments. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## Pipeline Testing

Hey, Cloud Gurus. Welcome to this lesson, Pipeline Testing. So by this point, we've done a lot with pipelines, and we've even talked about and done some manual tests where we don't want to fully automate our pipeline. We want to stop it and perform some manual tests. And certainly, that's a big part of our testing. But there are other types of tests we can do. And in this lesson, we'll give an overview of those tests we can do, where they are located, go into a little bit of detail on these tests and talk about their cost and their benefit. So in the lesson, we'll start with an overview, and this can also be a little bit of a review of what we've looked at with testing to this point. Then we'll detail all of the types of tests we can perform on our pipeline. And we're going to take a look at a testing pyramid, and this really boils things down from a cost and time perspective. And then we'll wrap up as always with some exam tips. So let's get started with the overview, and we start off with sort of a typical pipeline that we've seen before, source, build, and deploy. Maybe we're using CodeCommit and deploying our application to a web server. Now what we've seen is sure, it's nice to fully automate that pipeline. But what if we want to do a bit of testing? Well, we can add some manual approval stages. Now why would we do that? Well typically, maybe we had an issue and we pushed that issue out to our server out to our application. Maybe even in prod. That's not good. So in that case, we can add some approval stages, manually approve, and then move on from there and ultimately deploy. But let's get a little bit more specific about testing and talk about some of the tests that we can perform along our pipeline. So during build, we can perform unit testing. We can also perform a static code analysis. The unit test, as unit implies, tests a specific section of code and just makes sure that the code does what it's expected to do. Static code analysis also is pretty self-explanatory where we're just examining our code for errors, maybe some security holes, and just make sure that code adheres to our guidelines, maybe a code review. All right, moving to the right, think of this as our pipeline, moving to maybe staging or test area. And at this point, we can do integration testing. We can do component testing, systems testing, performance testing, compliance testing, and user acceptance testing. Our integration test verifies interfaces between components and against our software design. So we're integrating multiple components, making sure everything works as expected. Component testing, similar. We're testing messaging between various components and just checking the outcomes. Is everything as expected? System testing, we're testing the system end to end. And does this software satisfy our business requirements? Performance testing, self-



explanatory. We're putting a load on our system and making sure that it hits our performance targets. Think of like service-level agreements. Compliance testing, we have to meet certain standards with our application, and we're testing that. User acceptance testing, another one that's fairly self-explanatory. Does our system end to end meet our business goals? Our customers can get involved in alpha and beta testing, and that's part of our user acceptance testing. Keep moving further right in our pipeline. It looks very similar. We can perform all of these same tests. An additional test is canary testing. And remember those percentages. Maybe we send 90% of our traffic to our existing site, 10% of the traffic to our new build. See how things go. If things go well, we shift traffic to our new build. And again, a key point is all of the stages of testing that we see in staging and test can be performed on prod. All right, let's check out the testing pyramid, and this sort of is adapted from Agile methodologies and really one of the Agile gurus. Mike Cohn created this pyramid and some testing such as user interface, performance and compliance, service integration and component testing, and finally, unit testing at the bottom. Let's reveal more, and it'll make more sense to us. First of all, the lower we are at the pyramid, the faster these tests are. It sort of makes sense. Unit testing, think about it. We're not testing an entire system, just a unit of that system. And of course, UI, that's sort of the end product, the entire system, the user interface. So that's going to be slower. Well, guess what? Fast is cheaper, slow is more expensive. So we can do a lot more unit testing. It's cheaper. Well, guess what? Unit testing should make up roughly 70% of our testing. Sort of more bang for our buck. So again, we've talked a great deal about putting manual approvals into our pipeline. And typically, when we do that, we're going to test things out. And it's just important to have an overview understanding of testing, different types of tests that we can perform, where in our pipeline we perform them, and what they may cost. So let's look at some exam tips. Which type of tests should comprise most of our testing? Well, we know that, unit testing. Cheap and fast. Good rule of thumb, 70% of testing should be unit testing. Know which tests are performed at each stage of the development cycle. Example, 70% of our testing is unit testing. Well, where's that at that? The build stage, and finally, test stages can be added and removed from our pipelines. We've seen that already. We can always set up manual approval for safe error-free deployments. There's sort of that tradeoff. Automation is great, but we don't want to be deploying errors. There's certainly a time and a place for manual tests and setting up manual approval. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## CodePipeline Extras

Hey, Cloud Gurus. Welcome to this lesson, CodePipeline Extras. In this lesson, as the name implies, we're going to take a look at some extra techniques that we can use with CodePipeline and maybe have a different spin on some things we already know about CodePipeline. And we'll finish up with a short demo on how



to invoke a lambda function from CodePipeline. All right, let's first talk a little bit more about CodePipeline with EventBridge, and most of this is certainly review. So we have our EventBridge events, and those events are composed of rules. And a rule is configured by choosing an event pattern and a target, and the target will be a service. So if we're using EventBridge with CodePipeline, some of our targets are going to be the usual suspects such as Lambda, we can trigger lambda functions, and SNS, we can set up notification chains. Now a key point, EventBridge needs permission to invoke the selected target. So we're talking about policies attached to service roles. And the cool thing is these best practices, we'll see the repetitive nature, and we'll just know from experience what we need to do to get our services to interact with each other. So how does CodePipeline fit into this picture? Well, CodePipeline can report an event to EventBridge whenever the state of a resource in your AWS account changes. Think about the types of events we could be talking about, pipeline executions, stage executions, and action executions. And we'll get a visual refresher on these executions in CodePipeline in the upcoming demo. Let's go a little bit further into this. Continue talking about events, and using events with CodePipeline has come to the forefront. If you've worked with CodePipeline in the past, this might be a refresher as well. Back in the day, you could set up polling on your pipeline to periodically check for changes. Are you old enough to remember that? Probably just a couple years ago. But think about that, you're polling your pipeline periodically, whatever, every 5 minutes or so. Well, that's certainly not real time. Why not just make it event-driven using EventBridge? Exactly. And this is a key point. Polling has been disabled for inactive pipelines. If you have a pipeline that's running and it's using polling, oh well, you can keep using polling. If you want to be more efficient, you could certainly edit your pipeline or create a new event-driven pipeline using EventBridge. So we're done with polling or we should be done with polling. Let's go event-driven. And there still remain instructions to migrate polling pipelines to event-based. You may run into one of these. All right, what about webhooks? Probably if you work with CodePipeline at all, you've heard about webhooks. Let's look at a simplified pipeline. So here's a clue right off the bat. Think about our repositories, CodeCommit, S3, and outside of the AWS umbrella, GitHub. And of course, we can use GitHub as our repository in a pipeline, and we just need to be aware of other techniques to set up our event-driven pipeline. So our local devs are submitting code to GitHub, forming commits to be specific. What do webhooks do? Well, they help get us event-driven. Webhooks are the mechanism by which GitHub can notify your pipeline about such events. I like to call webhooks EventBridge for GitHub. Basically, again, it's helped making our pipeline event-driven. All right, let's talk about action types, and this is a different spin just looking at a pipeline a different way. We've seen pipelines with different stages and various providers. So for example, our source stage could be CodeCommit, S3, GitHub. And let's just look at this from the perspective of action types, and each action has action providers. So rather than stages, let's think about this from action types, and this



can be helpful. So we have a source action type, build, test, deploy, approval. And you'll notice that there are more action types typically than stages. What other action type is there? Invoke. Now we can still tie our services to action types. Think about source action types and S3, CodeCommit, ECR. What about build action types, CodeBuild, Jenkins, test, deploy? These are the providers for these action types now, approval, manual or automated. What about invoke? Lambda and Step Functions. So just another angle to look at our pipelines from. I do find it helpful to think about these action types. And of course, in grammar, these are verbs. All right, and let's drill down into the invoke action, and let's invoke something. We'll do this from the Management Console. An abbreviated demo, just want to hit on the key points. We've certainly seen enough pipelines by now. Now remember, invoke can be Lambda or Step Function. We're going to focus on a lambda function. Now when we invoke a lambda function from a pipeline, some of the use cases could be we can create resources using CloudFormation, perform a CNAME swap. Think about blue/green deployments. We could create an AMI. Let's wrap up by seeing what this looks like from the Management Console in CodePipeline. All right, so, I'm looking at a pipeline we've worked with in the course. Pretty simple. Our source stage is an S3 bucket, and we have some code stored in that bucket. So when our pipeline executes, that code is retrieved. And then using CodeDeploy, that code is deployed. Now what if we needed to invoke a lambda function? Well, I've done that. And don't be alarmed, this Abandoned here. I was just running tests. I abandoned that myself. But let me click Edit on this pipeline, and we can add stages, of course. And that's what I did. I added a stage, but I'll just show this. Add stage, click Add action group. And we have all of these action providers, and, of course, note our drop-down. Back to our diagram. Here are our actions, Approval, Build, Deploy, Invoke, Source, and Test. And of course, just select Lambda and a lambda function. Done that already, so I'm just going to delete this one, and let's check out the one I created. So, what the function does, not terribly important here because just showing the action type, Invoke, in our pipeline, we want to invoke a lambda function. That's what we want to accomplish. Whatever the Lambda function does could be anything under the sun. In this case, it's just a simple Hello World. All right, now one very key point about this lambda function, and I'll go over to Lambda, and here is the function. And I'm on the Configuration tab, ah, Permissions. I always have to remember that. I have an execution role, CodePipelineWithLambda, CodePipelineLambdaExecPolicy. Okay, makes sense. Our LAMBDA function, access, Limited:Write, CodePipeline and CloudWatchLogs. Just remember, pretty much always need that execution role. Let's go back to this pipeline. And real quick, let's wrap up. I'll click Release change to start my pipeline. It goes and grabs our source, deploys it. That succeeded. It invokes our lambda function, really simple. Just a Hello World function, but it could be anything. View details, View in Lambda. But notice, it actually takes us to CloudWatch to our log group. And yes, our function was invoked less than a minute ago. So that is a detailed look at the invoke action type in a CodePipeline using AWS Lambda. And with that, I'm going to end this



lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## AWS CodeArtifact

Hey, Cloud Gurus. Welcome to this lesson on AWS CodeArtifact. So in this lesson, we'll take an overview look at CodeArtifact and just define what is CodeArtifact and what can it do for us? And then we'll look at some diagrams and see how it fits into our pipelines. It will all make sense when we check out the diagram. We'll have a brief demo on CodeArtifact, and I say brief because there's also a hands-on lab for CodeArtifact where we'll go much deeper. And we'll wrap up with some exam tips. So with all that said, let's talk about AWS CodeArtifact. All right, what is CodeArtifact? Well, it is a software artifact repository. Well, let's put that into layman's terms. CodeArtifact is a software package management tool, and it can be used to store and provide software packages and libraries for use in our development and in our applications. Now when we think about packages, remember that product from CodeBuild, which are software packages. And yes, we can store those software packages in CodeArtifact, but those packages can come from many different places. Now, for in-house development, maybe we're doing Java, maybe we're doing .NET, specifically maybe C#, JavaScript, Python. Well, we're going to have artifacts from this development in the form of packages, and we can store those in CodeArtifact. Now I'm showing Maven for Java, Gradle for .NET, Gradle can work with multiple different languages, NuGet npm for JavaScript, and pip for Python. So that's in-house, but there's even more we can do. So we can link to public repositories. So CodeArtifact can act as a mirror for these public package repositories. And that really extends the power of CodeArtifact. So for now, just think of it as we are able to pull in packages from public repositories. Now let's go a little bit deeper on CodeArtifact and what are the key building blocks? Well, CodeArtifact is made up of two main components. The domain, and a domain is really just a wrapper that allows you to group together repositories. And as you might guess, that other main component is repositories. And this is where we store our packages. Now additionally, we can have upstream repositories, and we can connect to those. And that is how we can perform inheritance within our repositories. So, let's say we make this upstream connection and a developer is looking for a package in this number one repository. Well, guess what? If the package is not found here, CodeArtifact will look for the package here. And if it's there, well by inheritance, the developer can use that package. So we can specify one repository as being an upstream repository of another. And again, if it's in that upstream, because of that connection, we will have access through inheritance. Now let's just see where this fits in when we think about deployment pipelines. Well again, we're building packages in CodeBuild, and certainly we can store those in CodeArtifact. But let's not limit our thinking to just that. Again, we can have packages in-house, we can link to packages from public repositories also. So, as far as where this fits in, well, it's just the same as, hey, we build a



package in CodeBuild. Well, now we can store it in CodeArtifact. How do we use that? Well, we can pass it on to CodeDeploy for deployment, or we can deploy to Lambda. So CodeBuild and CodeDeploy can pull and use these packages from CodeArtifact. So, it's really very simple. Nothing magical about where this fits into our pipelines. We've already used packages built by CodeBuild. Well, this is just an extra step. This is where we store all of these packages that we create, and we can draw on those within our pipelines. So that is a look at CodeArtifact. Let's take a look at it from the Management Console. Again, I'm going to keep this demo brief. It will sort of be a segue into the hands-on lab on CodeArtifact. All right, at the Management Console, I just did a quick search. Let's go into CodeArtifact. And just notice, so we have our Source, CodeCommit, Artifacts, CodeArtifact, Build. Our build packages can be stored in our CodeArtifact repositories and the rest of our pipeline. Deploy can pull from CodeArtifact those packages and deploy them. And of course, CodePipeline is sort of the conductor of this orchestra, so to speak. All right, so let's just check out domains, and I will go ahead and create a domain. Give it a name. Notice it built a URL for us. Now we can do some KMS if we like. I'll just keep the default. Let's go and create that domain. All right, now likewise, remember the domain holds our repositories. Give that a name. Do we want any public upstream repositories? Ah, a lot we can choose from. Let's save that for the hands-on lab. Click Next, and go ahead and create our repository within our domain. So if we go back to Domains, there's a repository. All right, opened another tab, and this is sort of a sneak preview of CloudShell, which we'll talk about in another lesson. And this pretty much sums it up, browser-based shell with AWS CLI access from the Management Console. Now, key point here is we're already logged in, so we can open this shell. We're already authenticated. Probably go ahead and read this if you haven't seen this before. There's our command line. So let's just run a couple CodeArtifact commands. That's how we set up a CLI command for CodeArtifact. List-domains, we better have one when we run this. There we go. There's the domain we created from the Management Console. What about repositories? We also created one of those. And there we go, there's our demo repository within the domain we created. So that's a look at CodeArtifact from the Management Console and a little bit of the command line and a sneak preview of CloudShell. And we'll go much deeper on CodeArtifact in the hands-on lab. All right, let's take a look at some exam tips for CodeArtifact. So, maybe a good tip is to recognize the build tools associated with the different languages. Just have a good idea of how that all fleshes out. Certainly pip with Python, we're probably most familiar with that. Understand how CodeArtifact interacts with CodeBuild and where it fits in the pipeline. Well, we create packages in CodeBuild, we can store packages in CodeArtifact. It's just important to realize that we can get packages from all over the place and store them in CodeArtifact, including popular public repositories. And one more point, understand that repositories support inheritance, and as a product of that, a domain can provide deduplication of packages. In other words, if you're inheriting from a repository and you have duplicates of packages, well, the domain



recognizes that, performs deduplication, you only pay once for storing a package. So that is a look at CodeArtifact, and certainly check out the hands-on lab. We'll go deeper from the command line. But for now, I'll end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## Automating EC2 Builds Using EC2 Image Builder

Hey, Cloud Gurus. Welcome to this lesson, Automating EC2 Builds Using EC2 Image Builder. And with Image Builder, we can really start at the root or the foundation of our builds. And that is the Amazon Machine Image, the AMI. And with EC2 Image Builder, we can build pipelines to automate the build of our golden AMIs. And a golden AMI is an AMI that we can standardize, make sure it has consistent security patching and hardening, and that is the AMI that we always want to use. And then of course, we can further our automation by deploying this AMI maybe through CloudFormation templates or several different other ways we can automate our builds. So in this lesson, we will focus on EC2 Image Builder. We'll take an overview look at it. Talk a little bit about how it works. The real important part of this, this is a service that really, if we just get in and use it, we'll understand it. So we'll do exactly that with a demo, and then we'll go over some exam tips. So let's take an overview look at EC2 Image Builder. All right, with Image Builder, what can we do? We can create an automated pipeline to create custom images. We can also maintain those images. So if there are patches or updates that we would like, that can be automated as well, and we can test those images. Now there's a little bit of a surprise here. What kind of images can we create? Well, we can create EC2 AMIs. But wait, there's more, as they say on the commercials. We can create Docker images as well. So let's take a look at how it works. So we start with a base image, and I'm just going to focus on EC2 AMIs. But again, remember this could be Docker images. And we can customize that image. And we're working toward our golden AMI. That's the one we want to use. We can secure the image. Finally, we test the image, everything checks out. We can go ahead and distribute this image, and this is the image that will be deployed in our accounts. So this golden image we're working toward is produced on our schedule without having to write a single line of code. And again, we can do the exact same thing with our Docker images. And really, the best way to understand this service is to go ahead and try it. So let's go to the Management Console into Image Builder, and let's build an image pipeline. All right, so I'm logged into AWS, and I go into EC2 Image Builder. Let's create an image pipeline, and we can just click on this, Create Image pipeline. Now we have options here for how we want to go ahead and do this. We could do one manually or with a cron expression. Let's just stick with our schedule builder, and let's just stick with the default. So every Monday at 9, notice the time zone is UTC, our pipeline will run. Now we just want it to run every Monday at 9. What if there's nothing to update? Well, we can choose this one, Run pipeline at the scheduled time if there are dependency updates. So reading the finer print, we



just need to add versioning, and we'll do that momentarily. We have optional tags we could add to this. But before we do anything else, let's give our pipeline a name. I'll call it `CompanyGoldenImage`, and then we can just click Next to move forward. All right, so we need a recipe. Use existing recipe, well we don't have an existing recipe. Let's create one. All right, what type of image are we creating? Let's go with AML. But again note, we can do Docker. Give our image a name. We have to enter a version number. Don't be fooled by it, the grayed out. That's just a helper. It actually has to be entered. Optional description. All right, and just note the different places we can get these images. We'll stick with the default here. We want Amazon Linux 2. All right, so what is our baseline image that we're going to modify? So we go to Image name, drop-down. Let's start with Amazon Linux 2 x86. Let's keep Use latest OS version. We'll accept the defaults for instance configuration. Now, notice the default is to have the SSM agent, Systems Manager agent. The non default would be to remove it, but let's keep that. Notice for our image, we could add user data, and that's the same as if we were just creating our own image and maybe adding some user data. There's a working directory for use during build and test. What about components? Well, this is going to be our golden image, and we want to make sure to catch security updates. So, for the search, I put in the update keyword. I'm going to select update-linux. This updates Linux by installing all available updates via the UpdateOS action module, Owner. I'm going to move this over, Expand all. And just to see, let's stick with Use latest available component version. We could add some tests and maybe just browse this list, see what they do for us. Very informative. I'm just going to keep this simple though. All right, rest of the way, we'll just accept the defaults, and click Next. Again, this is informative reading. Maybe check this out, but we're going to accept the default configuration using service defaults, creates an IAM role and an instance profile. Let's go ahead and click Next. This define distribution settings is optional as well. One more Next, and then we can review what we're creating. Notice, Remove SSM After Build, Not enabled. The default is to have SSM on our image after build. We want that. All looks good. Let's click to create our pipeline. There we go. Our pipeline is enabled, and it's going to create the image that we specified during the creation of this pipeline, our golden image. And every Monday at 9 UTC, it's going to check for updates, patches, whatever that should be applied to our AML. If an update is necessary, our pipeline will run and create a new golden image. This golden image will be distributed throughout our organization. This is the one we used, perfectly hardened to our specifications and in compliance. All right, that's a look at creating an image pipeline using EC2 Image Builder to create our golden AMLs. To wrap up the lesson, let's take a look at some exam tips. What types of images can be customized? The name of this service is a little tricky. Yeah, we can certainly do EC2 AMLs, but we can also create and customize our Docker images. How can you automate updates and system patching for your images? We just did it by setting up a pipeline in EC2 Image Builder. We can check automatically, we can set up that schedule, or we can just check for updates on a scheduled time of week and only run the pipeline



if there are updates. What could be a key word on the exam to indicate the use of EC2 Image Builder? A golden image or a golden AML. And that is the image we standardize through configuration, consistent security patching and hardening. If we set up our image pipeline properly, that will all happen for us automatically. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## Amazon CodeGuru

Cloud Gurus, welcome to this lesson on Amazon CodeGuru. And CodeGuru is a service as it relates to the DevOps Pro exam. If we know what it is, what we can do with it, and where it fits into our deployment pipelines, we should be in good shape. So in this lesson, we'll take an overview look at Amazon CodeGuru, and then we'll start to talk about the components of CodeGuru, the Reviewer, the Profiler, and a really handy tool, the Secrets Detector, and we'll finish up with some exam tips. So let's go ahead and look at CodeGuru from an overview perspective. All right, so what is Amazon CodeGuru? CodeGuru is a developer tool that provides intelligent recommendations to improve code quality and identify an application's most expensive lines of code. And if we sort of read between the lines here, we could kind of guess where CodeGuru fits in. So, our developers write code, and that code is going to be committed to our repositories. We'll see that in an upcoming diagram. This is more of an overview look. The CodeGuru Reviewer can actually review this code. Code Reviewer uses program analysis and machine learning to detect potential defects that can be difficult for developers to find. And then it can recommend fixes in our code. Now, CodeGuru Profiler can operate at multiple positions in our pipeline. The Profiler provides interactive visualizations and recommendations that help us fine tune our application performance and troubleshoot operational issues. Now, this can happen in our build and test stage. It can also happen in our prod environment. So we write the code, we commit it. CodeGuru Reviewer can review that code. We make improvements, we perform tests. The Profiler can analyze this code for us. Ultimately, we deploy it, and we get into the prod environment, and the Profiler can analyze this code. And maybe a key term that we want to remember, you might see it on the exam, that it's associated with CodeGuru, the CodeGuru Profiler can help you find the most expensive lines of code. And what do we mean by expensive? Well, in terms of CPU usage or latency. And the Profiler can actually suggest ways to improve efficiency and remove bottlenecks. So again, it can do it back in build and test. But when we get to prod, it continues to do that. Maybe in prod, our workloads are different than our workloads in test and our traffic, and there's just more demand. It's more taxing on our applications. So from a high level, that's what CodeGuru can do for us. It's a developer tool that provides intelligent recommendations to improve code quality and identify an application's most expensive lines of code, really cool stuff actually. All right, let's focus on CodeGuru Reviewer. So again, the developers



commit the code. And in this graphic, let's say we're committing to CodeCommit. CodeReviewer goes into action. CodeReviewer can use machine learning, and automated reasoning, and also AWS security best practices. And remember, machine learning, it is learning. The more code reviews it's done, and it's done thousands, the better it gets. It detects defects and vulnerabilities. It can be difficult to find in our code, Java and Python, and offer recommendations on remediation. So it provides built-in reviews with actionable recommendations. So, CodeReviewer, as far as its place in the pipeline, think about when we commit code, we want to review. Reviewer, review that code. Let's talk about Profiler. All right, so we have our code. We committed it. It got through our code review, we did our improvements, we're ready to move on. So we build. CodeGuru Profiler can be used to detect and optimize expensive lines of code. So it's searching for application performance optimizations and recommends ways to fix them, and it can reduce CPU utilization or cut compute costs and improve our application performance. All right, so let's say we run CodeGuru Profiler. It gives us recommendations. We remediate that. We deploy out to prod. Well, guess what? CodeGuru Profiler can keep working for us. We get out to prod, it's a different ball game from maybe our test environment. More users, higher demands, higher stress on our application. And CodeGuru will just detect the anomaly is an expensive line of code and give us our recommendations, and it's an iterative process. So that's really CodeGuru in a nutshell. Just know what it can do for us. Know its two main parts, the Reviewer and the Profiler, although we'll see another one here momentarily, and where it fits in. We commit code, we can do code reviews with Reviewer. And when we build and deploy, Profiler can work for us before we get out to prod and after we get out to prod. Now one other cool, very helpful offering in CodeGuru is the Secrets Detector, and this is so valuable. So, we commit code. And again, we're using CodeGuru. It's going to perform our code review. Let's say we're using configuration files. And we should be past this, but let's say we're storing secrets in config files. It could be even a worst case scenario of storing secrets in code, which is absolute no, no. Well, CodeGuru Secrets Detector, what it can do for us, identify hard-coded secrets during code review, suggest remediation steps to store secrets in Secrets Manager. In other words, if we have secrets in our code or config files, it's going the point that out to us and strongly urge us to get those secrets into Secrets Manager. So let's say we come into a new organization. We know about Secrets Manager. We know about best practice, but we don't know if there are secrets in our code or in config files. Well, we can find out up front and sort of be the hero in this situation. Use Secrets Detector to get our secrets into Secrets Manager. That is a bit of the awesomeness of CodeGuru. Let's check out some exam tips. What does CodeGuru Reviewer do? It provides intelligent recommendations to improve code quality and identify the most expensive lines of code and help us remediate those issues before we deploy. What does Profiler do? Detects and optimizes expensive lines of code. Certainly at prod, but can also happen in our test environments. And we just want to understand where CodeGuru performs in our deployment



pipelines. Reviewer interacts with code when it is committed and performs code reviews. Profiler can interact during build and test and probably most importantly, after application deployment. The demands out at prod may uncover some extra issues. All right, so that is a look at Amazon CodeGuru. I'll end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## AWS Amplify

Hey, Cloud Gurus. Welcome to this lesson on AWS Amplify. And Amplify is a service that we really should just know what it is and what it does and maybe have a picture in your mind of where it fits into a DevOps process. All right, so what is Amplify? Amplify is a set of tools and features that enables front-end mobile and web developers to quickly and easily build full-stack applications on AWS. So it's sort of your one-stop shop to build full-stack apps on AWS. And we hear that term, full stack all the time or you will if you're in software any amount of time. What does that actually mean? Well, in the full stack, we have front-end development, back-end development, databases, DevOps, and mobile app development. So when using Amplify, you're going to have the tools available to you to develop and maintain this full stack of services. So Amplify provides all of the building blocks for all stages end to end in an application. How does Amplify work? Amplify provides the following products to build full-stack iOS, Android, Flutter, Web, and React Native apps. Amplify provides two services, Amplify Hosting and Amplify Studio. With Studio, you build your full-stack web and mobile app using a simple and intuitive visual development environment. So think of your IDEs, maybe Eclipse or IntelliJ for Java or Visual Studio Code for any number of languages. That's what we're sort of talking about here. And also work from the CLI. Configure all the services needed to power your back end through a simple command-line interface. And then there's a UI library to build front-end apps or React or React Native or Angular, Vue, and Flutter. There are also client libraries, common use cases, Auth, data, and file storage and help connect your front-end app to your back-end resources. And finally, there's Amplify Hosting. Now again, Amplify Studio and Amplify Hosting are the two main services, but certainly these other utilities are well worth mentioning. Amplify Hosting provides a git-based workflow for continuous deployment and hosting of full-stack web apps. So, if we think about Amplify Studio, analogous to one of our favorite IDEs, and Amplify Hosting, well git-based workflow, we're familiar with Git by now, we should be up and running with Amplify very quickly. All right, let's just wrap this up by getting an idea of where Amplify just fits into DevOps. Just have a mental picture of what this looks like, and that'll serve us well. So Amplify is on the front end, and we have Cognito, and we request and receive a token from Cognito. And what has triggered all this is a request from the outside. So we forward that request with our authorization token to API Gateway. Where do you think this is going? Well first of all, it's going to validate that token. API Gateway, as you've probably guessed, can form the request and trigger our lambda



function. And let's say that Lambda retrieves some data from our DynamoDB database. Now note the arrows going both ways. Lambda retrieves the data, and it's sending back the response back through API Gateway and ultimately back to our front end. So that gives us a look at where Amplify fits into a common scenario. It's really pretty straightforward. Again, it has utilities similar to our favorite IDE and can use git-based commands. And we can issue commands from the CLI as well. And in a sense, it ties everything together that we've learned up to this point. So that's a look at AWS Amplify. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

## SDLC Automation Review

### SDLC Automation Scenarios

Hey, Cloud Gurus. Welcome. So in this lesson, we're going to take a look at some scenarios in a different way. And at the end of each section of the course, we'll continue to do this. And the idea is to make it sort of a visualization exercise just to help us as we get ready for the DevOps Pro exam. We see those scenario questions and just to be able to picture a scenario in our mind's eye. So we'll talk a bit about what we want to accomplish with the lesson, and then we'll just look at four scenarios and give us some good practice leading up to the DevOps Pro exam. All right, let's talk about what we want to sort of accomplish here. So, we want to read the scenario and try to visualize it. Now, admittedly, these scenarios are going to be less complicated than the scenarios on the exam, but it's good to start small and work our way up. We'll certainly see some complicated scenarios in the practice exam. The key is we just want to try to visualize what we are reading and then secondly, sketch the scenario on a notepad, like one available on the exam. And then we'll click through the diagram and talk about the solution. Now, here's the catch. On the exam, we have roughly 2 minutes per question. And on the real exam, we won't have time to sketch every question. So the most important part of this is just the visualization of the scenario. And usually when I'm on the real exam, if I run into a difficult question, I don't waste a lot of time spinning my wheels on that question. I'll mark it for review, and those are typically the questions that I like to sketch out. Again, we won't have time to sketch out every question on the exam, just a select few that we find really difficult. And one thing I'll find is that I run into a difficult question, I mark it for review. And I'm trying to get some wins, catch some low-hanging fruit, get some questions that I know I got right, build my confidence up. When I get to these marked questions the second time around, almost always they seem a little bit easier for me. So with all that said, let's take a look at some scenarios. All right, what tools can we use to grant and/or restrict access to our CodeCommit repositories? So we have CodeCommit, and we want to restrict access. And at this point, just maybe pause and think about the scenario, sketch



it out, sketch out the solution, but I will go ahead and just move forward here. And what we want to do is create policies that will grant levels of access to CodeCommit such as admins having more permissions than regular users. How can we grant developers full access to CodeCommit, but not the ability to create and delete repositories? And if you think back, we did see this in one of the lessons, and that is AWS CodeCommit Power User. And what we could do is create a group in IAM, attach this policy to that group, and place our administrators in that group. Let's look at another scenario. Where would we store CodeBuild artifacts, and what do we need to do to prepare the storage location? So we have CodeBuild. And maybe pause here and just visualize the scenario and maybe sketch out a solution, and I will go ahead and proceed now. So we want to store our CodeBuild artifacts in S3. And some actions we want to perform on S3, think about that. What are those actions? We want to turn on versioning. Remember, we may have multiple versions of our code. And really anytime we're using S3 as a code repository, we turn on versioning. And certainly for security purposes, turn on encryption. Let's check out another scenario. We need CodeBuild to run after code is committed to the CodeCommit repository. What steps can we take? So this one has a few more moving parts. Try and visualize that and then sketch it out. So we have CodeCommit, our developers are committing code. What we can do is have a handy EventBridge event, and that EventBridge event is monitoring CodeCommit for those commits by our developers. And the event can be a trigger for CodeBuild. The code is committed, EventBridge picks that up. It's actually sitting there waiting for it, triggers CodeBuild, and builds the code. Let's take a look at one more scenario. So we have a code pipeline consisting of CodeCommit, CodeBuild, and CodeDeploy. So visualize that in your mind, sketch it out. We want to perform a code review before building the code, as well as do QA testing before deploying the code. So visualize that and add that to your sketch. What steps can we take? So we have CodeCommit. We want to perform a code review before building the code. So we can add a manual approval to our code pipeline between our code and our build stages. And we go ahead, and we perform a code review, manually approve. That restarts our pipeline. We can do the build. So we do the build, but we want to perform QA testing before deploying. We can add another manual approval. Our pipeline stops. We perform QA testing. QA team says it's good to go, approves. We go ahead, our pipeline continues, we deploy our code. So we can add manual approval actions after the CodeCommit and CodeBuild stages. And the point is we can have more than one manual approval in our pipeline. So those are some scenarios for this section. And again, throughout the course, when we complete a section, we'll go ahead and reinforce what we learned by going through these exercises at the end of the section. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next section.

## Supporting Services for SDLC Automation



Welcome to this lesson, Supporting Services for SDLC Automation. And by supporting services, we're talking about services, which might not play a huge role when you take the exam, but they are mentioned in the exam guide. So it's just good to know what they are, what they do, and what services they can interact with. Maybe a good analogy would be they are supporting characters in a movie. So, some of these supporting services, we will talk about AWS App Runner, and we'll put App Runner into context and compare it to a couple of other services that perform similar functions. Then we will talk about AWS CloudShell. Now this next one might be a little bit borderline. It could be argued that CodeStar is not a supporting service. It's sort of an umbrella service for CodeCommit, CodeBuild, CodePipeline and a bit more, which we'll talk about shortly. Just think if you understand the fundamentals of our pipelines with CodeCommit, CodeBuild, CodeDeploy and just know where CodeStar fits into that scenario should be fine for the exam. And we will finish up with some exam tips. So let's go ahead and start talking about AWS App Runner. What is AWS App Runner? It is a fully managed container application service that lets you build, deploy, and run containerized web applications and API services without prior infrastructure or container experience. Now that certainly sounds a little bit like Fargate. And another analogy I'd like to make, sort of maybe a container version of what Elastic Beanstalk can do for us. Key point in this is fully managed. The inner app lets you build, deploy, and run containerized web applications. But behind the scenes, AWS is managing our infrastructure. Does this remind you of another service? I gave that away. It should, and that is Fargate. We'll talk about that difference here and then graphically just further down the page. So what's the difference? Well, it has to do with concurrency, working on multiple things at once. And the primary dimension used for scaling in App Runner is concurrency. So, you specify how many concurrent requests you want a particular instance of your app container to receive. Now where have we seen concurrency in AWS? Well, several places. The one that comes to mind is Kinesis and shards, certainly different services. but similar concepts. So, we definitely have this App Runner, Fargate similarities. All right, let's talk a little bit more about those differences between Fargate and App Runner, and we're going to add a third service. Think about what that might be. All right, Fargate leaves the concurrency and the scaling up to you. And like I said, let's add a third container and just give a high-level comparison of the three. So, sort of at the top we have Kubernetes. It's the hardest to configure, but the most customizable. I guess maybe analogizing a bit, remember that CloudFormation, OpsWorks, Elastic Beanstalk comparison. So, in that sense, this Kubernetes would be similar to CloudFormation at the top of that, maybe difficulty scale. Then we have Fargate. So Fargate's easier to configure than Kubernetes. I should also add maybe a little harder than App Runner, but it is less customizable. And then we have App Runner. It is the easiest to configure, but the least customizable. And again, jumping back to that CloudFormation, OpsWorks, Elastic Beanstalk analogy. This would sort of be at the level of Elastic Beanstalk. All right, so that's a look at App Runner. Again, sort of a supporting character in this exam. Know what



it does, where it fits in, and maybe how it compares to Kubernetes and Fargate. So let's move on and talk about AWS CloudShell. Now CloudShell has been out for a couple of years, but I think for the first time turns up in the new DevOps exam guide. So, CloudShell is a browser-based, pre-authenticated shell that you can launch directly from the AWS Management Console. I'll include an external link, but just going to go ahead and talk about it here. Then we'll do a quick demo. You can run AWS CLI commands against AWS services using your preferred shell such as Bash, PowerShell or Z shell. Key points here, browser-based. So, we don't need an EC2 instance. We don't need to SSH or instance connect to that instance if we have a browser and we have an AWS account. So we log into our account, Management Console. We go to CloudShell, we can just start using it. Now a real key point about CloudShell. At the time of this recording, it is supported in 12 AWS regions. Try to name a few here, US East, North Virginia, Ohio, Northern California, Oregon, Sao Paulo, Frankfurt, Ireland, Mumbai, Singapore, Sydney, Tokyo, and Seoul. And certainly with that link provided, you can keep abreast of its availability in regions. CloudShell also comes preinstalled with the AWS CLI, Elastic Beanstalk CLI, ECS CLI, and the SAM CLI, and runtimes, Node.js, SDK for JavaScript and Node.js Python, and the SDK for Python. So it's pretty handy if you want to very quickly run some CLI commands. And with that said, let's do just that. And let's do a quick demo on CloudShell. All right, I'm logged into the Management Console, and I go to CloudShell. Some good reading here. I preinstalled popular tools, talked about that. You have the same credentials as you use to log in to the console. Let's close that. There's our command line. So what I'm going to do is if you've ever worked with DynamoDB, a lot of the samples involve creating a table for music. So I'll go ahead, paste that in. So, this is just a CLI command, `aws dynamodb create-table, --table-name Music, --attribute, --key-schema, --provisioned-throughput`. Pretty simple table, but just a little quick demo on CloudShell. Let's run this. And getting back JSON from the CLI is more often than not good news. Let's wrap up our little demo by trust, but verify. Let's go over to DynamoDB. Let's go over to Tables. There's our Music table. So that's a look at AWS CloudShell. Certainly, if you need to run CLI commands quickly, this is a great option. Another thing is that you don't have to spin up an instance, and you don't have to pay for an instance. So it's there if you need it in a quick pinch. Certainly very, very handy. All right, let's talk about CodeStar. And again, this is kind of borderline between being a main character and a supporting character. I just think the fundamentals of our pipelines is really important. If you understand what's going on with CodeCommit, CodeDeploy, CodeBuild, even CodePipeline, know what CodeStar does, you'll be in good shape. So what is CodeStar? A cloud-based development services that provide you the tools you need to quickly develop, build, and deploy applications on AWS. So, maybe the key term here is develop, and we'll get into that momentarily. You can use a variety of project templates to start developing applications on EC2, AWS Lambda, and Elastic Beanstalk. Set up your entire continuous delivery toolchain in minutes. So, if you're starting to see some similarities between CodeStar and CodePipeline, well, you're on the right track. CodeStar connects CodeCommit or



Git, CodeBuild, CodePipeline, and CodeDeploy together using pre-defined templates. So, CodePipeline sort of oversees this deployment pipeline. And it's almost like, hey wait a second. It sounds like CodeStar is doing the same thing, and it in a way is. It's sort of an umbrella over all of this. I like to think of it as a dashboard as well. You can jump in and get started very quickly. Now remember the initial description mentioned development. And Cloud9 can be integrated and used in CodeStar as well. So there's that development aspect, and maybe that certainly differentiates it from CodePipeline, sort of an overlay to these underlying services and even CodePipeline being one of those underlying services. And if you understand these and deployment pipelines, you'll be in good shape. Just know where CodeStar fits in. So it's an umbrella over all of these. All right, let's talk about some exam tips for these SDLC supporting services. Which is more customizable, Kubernetes, Fargate or App Runner? I certainly know that, Kubernetes. It also takes more work, a little bit harder to use than the other two. And again, I like that analogy. Kubernetes is sort of to CloudFormation as Fargate is to OpsWorks and App Runner is to Elastic Beanstalk. Harder, more customizable at the top, less customizable, but easier to use at the bottom. Which is easiest to use? App Runner. How is AWS CloudShell preauthenticated? Well, we have to have our AWS account. Once we log in, CloudShell uses those credentials. In CodeStar, what allows you to start working with EC2, Lambda or Elastic Beanstalk quickly? Project templates. So that is a look at supporting services for SDLC automation. I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## DevOps Conversations for SDLC Automation

Hey, Cloud Gurus. Welcome to this lesson, DevOps Conversations for SDLC Automation. This is a new type of lesson I wanted to try out. And also, there's another type of lesson we'll be seeing. It's a word and term matching game. And my idea behind this is it's really good to have the fundamentals on some of these key services. Oftentimes on the DevOps exam, if you can pick out a key word or a key phrase in a scenario question, you can really sort of short circuit the question. You can eliminate two or even three answers. And let's talk about an example. Let's say we have a scenario question that says we need to deploy stacks in multiple regions and multiple accounts. Well, that is screaming CloudFormation stack sets. So I quickly scan the answers. And if it doesn't have stack sets in it, I'm not wasting any more time on that answer. So I can eliminate again one or two or three answers just by knowing some of the key fundamental aspects of these services. So in this lesson, we're just going to sort of look in on a conversation. And in this case, the scenario is an interview for DevOps role. Of course, we have the interviewer and we have the candidate. So let's go ahead and see how this goes down. All right, let's talk deployment pipelines. We're using CodeCommit, CodeBuild, and CodePipeline. And of course, CodePipeline is orchestrating this whole thing. How could we automate the pipeline when



changes occur in the main repository? So again, just trying to latch on to some key terms or words. Automate and automating the pipeline is jumping out at me. I'm thinking CodeCommit. How do we automate that? Here's the candidate's answer. There we go. That's how we automate it. You could set up an EventBridge rule to detect commits to the main repository, and this acts as a trigger to the pipeline. So, from this key term, automate the pipeline, should get in your mindset, okay, CodeCommit. How do we automate that? EventBridge. Scenario question on the exam, start scanning those answers for CodeCommit and EventBridge. All right, the interview continues. "Regarding CodeBuild, could I include an AppSpec file as part of the source code?" Well, key term here, AppSpec file. What does that belong to? Hint, hint, it doesn't belong to CodeBuild. Candidate is pretty sharp. "Well, the AppSpec file was used with CodeDeploy." So, on a scenario question, we see AppSpec file, we need to be thinking CodeDeploy. Scanning those answers for CodeDeploy, eliminating the ones that don't have it. Candidate goes on. "CodeBuild uses a buildspec file." There's that association. If we saw buildspec, then we're thinking CodeBuild. "But yes, the buildspec file can be included with the source." Interviewer, "Between the buildspec and AppSpec files, which has lifecycle hooks and how are these hooks used?" So, thinking about it, Auto Scaling groups have lifecycle hooks, and one of these has lifecycle hooks. Let's see what the candidate says. "The AppSpec file has lifecycle event hooks. They allow you to perform actions such as running scripts during different stages of your deployment." So I guess the key term here would be lifecycle hooks or lifecycle event hooks. If you see lifecycle hooks, you're thinking auto scaling, lifecycle event hooks. Think about the AppSpec file. Scan your answers for an AppSpec file. Eliminate the ones that don't have it. All right, let's continue. All right, interviewer, "With CodeDeploy, what are the deployment targets that we can use?" So, where can we deploy to? So I'm thinking, okay, EC2 instances. Where else can we deploy? A couple more. Think about that. Sort of the key term here, deployment targets. Our candidate is pretty sharp. What do they answer? "You can deploy to EC2 or on-premises servers, ECS, and Lambda." This candidate's on track for an offer here. All right, "For these different targets, are the hooks that are available in the AppSpec file the same?" This goes a little bit deeper. "No, EC2 has the most hooks available. followed by ECS, and Lambda has the least hooks available." And that's just a matter of glancing at that page in AWS on the AppSpec file and the hooks available, but they are different. All right, interviewer is sort of switching context here, trying to get the candidate off guard. "Is my pipeline, deployment pipeline, limited to just AWS tools?" "No, third-party tools can be integrated in a pipeline." All right, the interviewer is going deeper. "Can you name a few tools, one for the source stage and one for build and deploy?" So this is general pipeline 101 stuff, fundamentals that can really help us on the exam. Candidate's answer. "Sure! GitHub can be used as a source repository, and Jenkins can be used in the build and deploy stages." All right, so the interview just goes a little deeper on this topic. "Can I use S3 as a repository?" Candidate, "Yes, S3 can be used as a repository." I have a feeling this is going to dive a little deeper. "Can I



use S3 as a repository right out of the box?" "Pretty much." Although the candidate indicates a catch here, "You would just have to turn on versioning for the bucket." Repos need that versioning. We need to turn that on on S3. Our candidate crushed that, and our interviewer is happy with how this has gone. But again, just a different angle and a different way to look at things, sort of knowing those fundamentals because they can really help us eliminate incorrect answers on the exam. So thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

## Deployment Strategies and Test Scenarios

Hey, Cloud Gurus. Welcome to this lesson, Deployment Strategies and Test Scenarios. And we're going to take a look at various types of scenarios we may run into on the exam, and you'll certainly see a pretty wide range of different scenarios on the exam. And the idea is, yeah, we have scratch paper, we can sketch out scenarios, but we don't have time to do that for every question, and it sort of becomes a visualization exercise. So as we go through these scenarios, just take the time to visualize that scenario in your mind, and then we'll see where we turn out. So, we'll start simple, simple S3 deployments. We'll take a look at ECS deployments, Elastic Beanstalk deployments, Lambda. Sort of a sidebar, CodePipeline action types, but it kind of leads into other deployment types, and we'll see that. We'll look at CloudFormation deployments, and we'll wrap up with some exam tips. So let's start by looking at S3. And I've intentionally kept these use cases a little bit vague, and we just want to use our imagination and picture a scenario. It's okay if you don't get it perfect. Your visualization is going to help you and save you some time on the exam. All right, so we have a use case, very vague, deploy a simple application from an S3 bucket. So think about what that looks like. We'll talk about it a little bit here. All right, any questions? Well sure, several questions. First of all, where are we deploying to? All right, so we have our S3 bucket. Remember the key with S3 buckets. We want to turn versioning on. CodeDeploy is going to grab our code and deploy it somewhere. Again, where are we deploying to? Well, where can CodeDeploy deploy to? EC2, ECS, and Lambda. All right, let's talk about some key points. All right, so very basic. It's mandatory to have at least two stages in our pipeline. Our deploy stage is deploying an application, and I like to think of these targets as our hardware. And yes, you know Lambda is serverless, but there's some hardware somewhere. Remember to turn on your bucket versioning. Do you use S3 as a repository? And remember the three deployment groups to which we can deploy, EC2, ECS, and Lambda. So if you see a scenario with S3 as our source, this is sort of the picture you want to have in your mind. All right, let's talk about ECS. Again, a simple use case. We want to deploy a container to ECS. What does that look like? Think about that in your mind. What are the components we need? All right, in this case, we're using CodeCommit. Certainly don't worry about it if you thought of S3 for your source. All right, and then we're going to have CodeBuild,



CodeDeploy, and ultimately, we are deploying to ECS. Now we see an S3 bucket. Okay, what's that all about? And ECR, what do we need those for? Well, remember our code artifacts. Anything else? So, this is pretty vague, but let me expand upon that. Anything else related to CodeBuild and CodeDeploy? Do we need anything else to make this deployment happen? Another hint, are there some files that we need? Think about that. All right, CodeBuild, we need a buildspec. What about CodeDeploy? We need an AppSpec. So, these are visualization exercises, and certainly don't fret if you don't get everything perfect. All right, where can we test this? Well, we can test the multi stages of our pipeline. What if we need a code review? We could put a test stage in and grab that code from CodeCommit and do code review. We could test our build artifacts. Remember they're hanging out down here in S3, or we could do a manual approval before we deploy. Our artifacts can hang out in S3. We can put those in ECR, and, of course, CodeDeploy can deploy to ECS. So, probably didn't visualize all of that, but just want to make sure you're on the right track with your visualization. Remember as far as testing goes, we could also create a separate test pipeline, test there, then merge our code back and ultimately deploy. All right, let's look at Elastic Beanstalk. So, simple use case, we want to deploy an application to Elastic Beanstalk. What does that look like? Think about that, visualize it. All right, in this case, we're using an S3 bucket. We have CodeBuild in our pipeline, CodeDeploy, ultimately, Elastic Beanstalk. And maybe that raises a red flag with you. Oh, wait a second here. What is this? I thought we could only deploy to EC2, ECS, and Lambda. Well, that's where it's helpful to think of these as your hardware. You're deploying to this hardware. But remember, we deploy applications with Elastic Beanstalk. And remember, Elastic Beanstalk, if you're working in Elastic Beanstalk, you have the opportunity to configure your hardware when you deploy an Elastic Beanstalk application. So think of the application being housed somewhere here. And remember, EC2, ECS, and Lambda are the deployment targets. And I like to think of those, again, as the hardware. But we can deploy to Elastic Beanstalk, our application, and underneath is that quote, unquote "hardware". So, EC2, ECS, and Lambda, make sure to get this straight, this is important, are our deployment targets, and on top of that can have deployment providers. Elastic Beanstalk is one of several deployment providers. Where do we get a good list of that? I harp on this, but it's so true. Go into the Management Console build pipelines, study those drop-downs, and you'll know your deployment providers and just crush that on the exam. Okay, deploy a lambda function. That's pretty vague. And that's the idea again. Picture that in your mind. All right, so we start out. We have CodeCommit, CodeBuild, CloudFormation. Remember that connection between CloudFormation and Lambda. Ultimately, we deploy to a lambda function or deploy a lambda function. So in this case, the deploy stage is handled by, well, CloudFormation. CloudFormation is the deployment provider. You'll see that in your drop-down when you play around with your pipelines, and we are deploying a lambda function. Again, note that difference. Deployment provider, in this case, is CloudFormation. But our deployment group is a lambda function. I know that's



serverless, but it helps me at least to think of that as our hardware. All right, now a short sidebar, and we'll see where this goes shortly. Let's talk about CodePipeline action types, the actions within our pipeline. So we have source, build, deploy, okay, test, approve. That makes sense. All right, what is the sixth action type? And if this trips you up, well, that's the way to learn. You'll know now. What is the sixth action type? Invoke. What does that word make us think of, invoke what? Invoke a lambda function. So, within our pipeline, we can invoke a lambda function. Of course, with a lambda function, we can pretty much do anything under the sun, but what are some of the use cases? Well, we could create resources on demand. Maybe we need a new server based on what's going on in our pipeline. This can help with zero downtime Elastic Beanstalk deployments. So we could swap CNAMEs. Think of a blue/green deployment. A lambda can do that for us. We can deploy ECS Docker instances. Again, sort of creating resources on demand. We can back up our resources before deploying. So those are great use cases for invoking lambda functions. All right, let's talk about CloudFormation deployments. Deploy to CloudFormation. What does that look like? Say we start out with CodeCommit, CodeBuild, CodeDeploy. Ultimately, we're deploying a CloudFormation stack. All right, so obviously, we have a template. We need a template to deploy its stack. That's the only way it happens. How can we validate CloudFormation templates before deploying them? Think about that one. Well, we could use cfn-lint to validate our template. That is a tool for validating templates. How does this all come about? In the buildspec file. In CodeBuild, we can launch cfn-lint. So the deployment is handled by CodeBuild. And once the templates are validated, the stack can be created. All right, and a big part of this lesson really is visualization. In the exam, you're going to see several different types of deployment scenarios, and it's helpful to be able to visualize those as you're reading the scenario question. Your visualization doesn't have to be perfect, but if you get the major points right and certainly know the fundamentals such as buildspec, and AppSpec, and our deployment groups, and deployment targets, you'll be well on your way to passing the exam. And I harp on this, what's the best way to study pipelines? Create them in the Management Console and study those drop-downs of each stage for your available options. And then when you see something a little bit off the beaten path, maybe an ECS deployment or a Lambda deployment, you'll be ready. How can we swap CNAMEs in an Elastic Beanstalk deployment? We can invoke a lambda function in our pipeline. What deployment groups can we create in CodeDeploy, EC2 or an on-premises server? ECS and Lambda. Those are your deployment groups. So that's a review of deployment strategies and test scenarios. I'd more like to think of it as a reinforcement of the fundamentals. And if you get these fundamentals, it'll serve you very well on the exam. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.